

GEXLens — Manuál pro správce a vývojáře

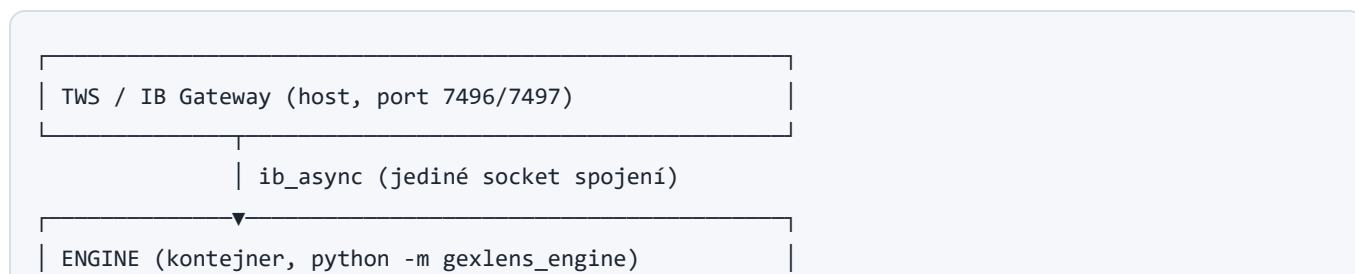
Verze 1.9 · září 2026 · interní dokumentace — není dostupná v aplikaci

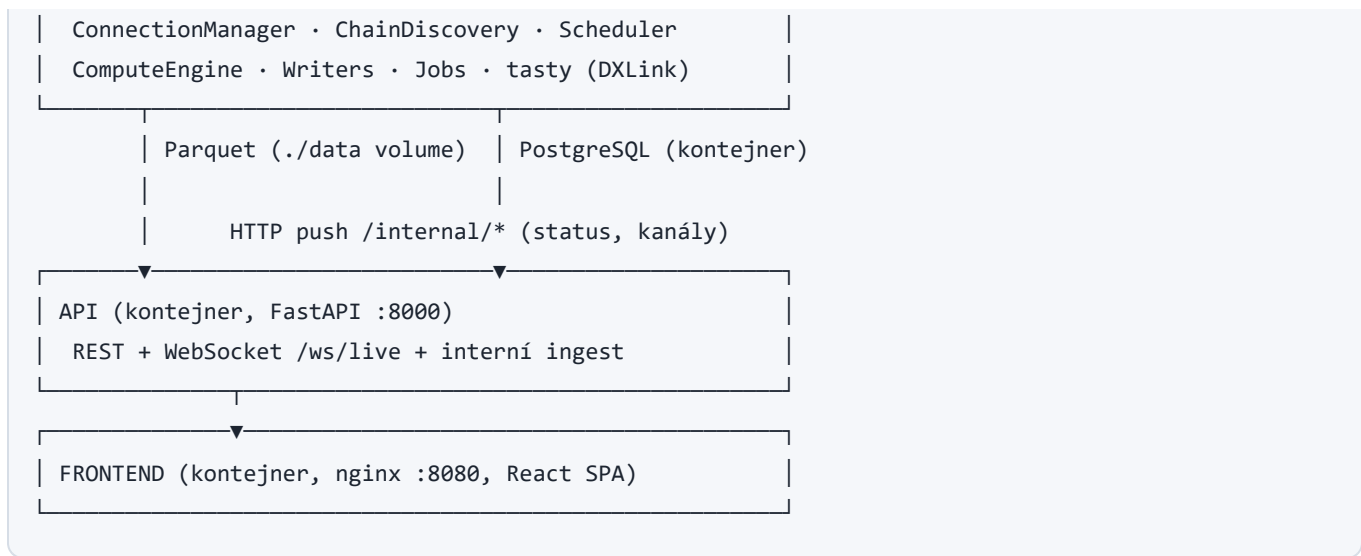
Technický popis architektury, provozu, konfigurace a vývoje aplikace GEXLens. Uživatelská příručka: `UZIVATELSKY-MANUAL.md`. Zdroj pravdy funkčních požadavků: [docs/SPEC.md](#) (v2.0); architektonická rozhodnutí v [docs/adr/](#).

Obsah

1. [Architektura](#)
 2. [Struktura repozitáře](#)
 3. [Provoz \(docker compose\)](#)
 4. [Konfigurace — kompletní reference](#)
 5. [Engine — datová pipeline](#)
 6. [Datové formáty a persistence](#)
 7. [API reference](#)
 8. [Frontend](#)
 9. [Vývojové prostředí](#)
 10. [Testy a CI](#)
 11. [Známé limity účtu a otevřené body](#)
 12. [Diagnostics a údržba](#)
 13. [Zprovoznění od nuly — IBKR účet, TWS/Gateway](#)
 14. [Bezpečnost a nasazení na server](#)
-

1. Architektura





Klíčové vlastnosti:

- **Engine a API jsou oddělené procesy.** Engine počítá a zapisuje; API jen čte storage a přeposílá live push z enginu (interní HTTP ingest → StatusStore + LiveHub → WebSocket klientům).
- **Vše lokální** — API CORS povoluje jen `localhost / 127.0.0.1`; žádná telemetrie.
- Engine se z kontejneru připojuje na TWS na hostiteli přes `host.docker.internal`.

2. Struktura repozitáře

GEX/	
└ engine/	Python 3.12 balík gexlens_engine
├ └ src/gexlens_engine/	
├ └ config.py	Pydantic Settings (.env, GEXLENS_*)
├ └ ibkr/	connection, discovery, scheduler,
├ └ └	underlying (bary+pacing), mock (pro testy)
├ └ compute/	gex, levels, heatmap, walls, cumdelta, profile
├ └ storage/	parquet_store, oi_archive, retention, meta
├ └ adapters.py	produkční ib_async adaptéry + HttpPublisher
├ └ runtime.py	EngineRuntime – minutový cyklus (testovatelný)
├ └ └ __main__.py	vstupní bod: discovery→archiv→smyčka
└ api/	Python balík gexlens_api (FastAPI)
├ └ src/gexlens_api/	main (routy+WS), data, heatmap (vektorizace),
├ └ └	live (hub), status, crud, alerts, meta_repo
└ frontend/	React + TypeScript + Vite
├ └ src/	components/, heatmap/, replay/, panels/,
├ └ └	profile/, annotations/, state/, api/
└ docs/	SPEC.md, adr/, manual/
└ docker/	Dockerfiles + nginx.conf
└ compose.yml	celý stack

└ scripts/	bootstrap, start skript pro plochu
└ Makefile	test / run / run-api / run-frontend / run-engine

Pravidla vývoje jsou v [CLAUDE.md](#) : práce po GitHub issues, golden testy výpočtů, IBKR se v CI nikdy nevolá živě (mock vrstva `engine/ibkr/mock.py`), komentáře česky / identifikátory anglicky.

3. Provoz (docker compose)

```
docker compose up -d --build    # start / rebuild
docker compose ps              # stav služeb
docker compose logs -f engine  # živé logy enginu
docker compose stop            # zastavení (data zůstávají)
docker compose down            # odstranění kontejnerů (volume pgdata zůstává)
```

Služba	Port (host)	Poznámka
frontend	8080	nginx, SPA + <code>/manual/</code> wiki
api	8000	FastAPI, OpenAPI na <code>/docs</code>
postgres	55432	⚠ záměrně ne 5432/5433 — na vývojovém PC běží nativní PostgreSQL na obou
engine	—	bez portu; TWS přes <code>host.docker.internal:7496</code>

Data: Parquet v `./data` (bind mount, sdílené engine↔API), PostgreSQL ve volume `pgdata` .
 Zálohovat stačí `./data` + `pg_dump` (hlavně tabulku `oi_eod` , která se nikdy nemaže).

Provozní detaily kontejnerů

- Kontejnery běží pod **UID 10001** (`docker/entrypoint.sh`) — bind-mount adresáře musí být zapisovatelné pro tento UID.
- nginx frontendu proxuje** `/api` na službu API — port API se ven nepublikuje. Adresa služby `api` se **resolvuje za běhu** přes Docker DNS (`resolver 127.0.0.11 valid=10s` + proměnná v `proxy_pass` , #993, v1.3): se statickým `proxy_pass http://api:8000/` si nginx IP vyřešil jednou při startu a po restartu Docker Desktop, kdy api dostal jinou adresu, vracel 502 na všechno (REST i WS), dokud se frontend ručně nerestartoval. Nově se proxy zotaví do ~10 s bez zásahu. S proměnnou nefunguje odříznutí prefixu koncovým lomítkem, proto je v konfiguraci `rewrite ^/api/(.*)$ /$1 break` .

- **Cache hlavičky (#858):** `index.html` , `/manual/` a SPA fallback jdou s `Cache-Control: no-cache` , hashované `/assets/` jako `immutable` . Do té doby si prohlížeč sám určoval, jak dlouho `index.html` podrží, a zastaralá kopie držela uživatele na starém bundlu i po deployi — hard reload (Ctrl+Shift+R) je od té doby jen záchranná brzda, ne standardní krok.
- **scripts/ je v image engineu (#960,** `COPY` až za `uv sync` , aby úprava skriptu neshazovala cache vrstvy se závislostmi) — dokumentované kroky po nasazení (`docker compose exec engine python scripts/...`) fungují bez `docker cp` .
- **Start skripty ukazují skutečný výstup dockeru (#975):** `start-prod.ps1` i `start-dev.ps1` pouští docker přes sdílený `scripts/_docker.ps1` — výstup teče živě a při selhání se k chybě přiloží posledních 25 řádků; preflight rozliší „docker není v PATH“ od „démon neběží“. Dřív každé selhání skončilo domněnkou „běží Docker Desktop?“.
- Shellové skripty mají v `.gitattributes` vynucené `eo1=lf` — checkout na Windows je nesmí konvertovat na CRLF (kontejner by je nespustil.)
- **Zaručený exit engineu (#779, v1.2):** fatální výjimka v `main()` končí okamžitým `os._exit(1)` — kontejner spadne a `restart: unless-stopped` ho zvedne. Dřív po pádu zůstal zombie proces (kontejner „Up“, mrtvý engine); při podezření na viselce: `docker kill -s USR1 gex-engine-1` vypíše zásobníky všech vláken (#771).
- **Broker news pásky (#734, v1.2):** kořeny se změřeným Error 200 (`BRFUPDN` , `DJ` — `DEAD_TAPES` v `ibkr/newsticks.py`) se přeskakují; Dow Jones broad tape neexistuje v žádné variantě, reálně tečou BRFG a DJNL.
- **Vyhodnocení shadow porovnání:** `scripts/feed_comparison_report.py` `[--days N]` `[--symbol ES]` `[--sessions 2026-08-14,...]` — agregace v DB, rozpad per podklad, filtr čistých seancí (vstup prahů #614). **Surová historie do 22. 8. je od 1. 9. zhuštěná** do `feed_comparison_daily` (#965, varianta B): 25,7 M řádků / 2,5 GB = 79 % databáze se zálohovalo při každém dumpu. Percentily nejsou skládatelné, proto tabulka nese dvě úrovně — denní řádky (`session_date`) a **celkové řádky spočítané ze surových dat** (`session_date IS NULL`), takže čísla citovaná v #614, #616 a #517 A zůstávají ověřitelná; nový řez (jiné okno, jiná podmnožina seancí) už nejde. Postup `scripts/feed_comparison_compact.py` `--build` → `--verify` → `--drop` (drop odmítne, když ověření neprojde). Databáze 3 169 → 653 MB; živý crosscheck na tabulce nezávisí (počítá se z `compare_minute`).

4. Konfigurace — kompletní reference

Zdroj: proměnné prostředí `GEXLENS_*` a `.env` (viz `.env.example`). Validuje se při startu — nevalidní hodnota = engine odmítne nastartovat se srozumitelnou chybou.

Proměnná	Default	Význam
GEXLENS_IBKR_HOST	127.0.0.1	V compose přepsáno na <code>host.docker.internal</code>
GEXLENS_IBKR_PORT	7496	7496 live / 7497 paper (TWS); 4001/4002 (Gateway)
GEXLENS_IBKR_CLIENT_ID	1	
GEXLENS_MARKET_DATA_TYPE	1	1=live; delayed engine odmítá
GEXLENS_CONNECT_TIMEOUT_S	10	
GEXLENS_RECONNECT_BACKOFF_BASE_S / _MAX_S	2 / 60	Exponenciální reconnect
GEXLENS_HEARTBEAT_INTERVAL_S / _TIMEOUT_S	30 / 15	Heartbeat spojení; agresivnější hodnoty vedly k falešným reconnectům během sweep dávek
GEXLENS_RECONNECT_STALL_ALERT_S	300	Watchdog reconnectu (#770): po tolika sekundách bez spojení alert <code>connection_stall</code> do zvonečku, opakovaně dokud spojení chybí; <code>/status.connection_offline_for_s</code> nese délku výpadku (klíč chybí, když spojení drží)
GEXLENS_SYMBOLS	ES	Základní sada futures podkladů (čárkami); watchlist z DB se přidává za běhu (ADR-0003)
GEXLENS_MAX_INSTRUMENTS	3	Strop souběžných instrumentů (rozpočet market data lines)
GEXLENS_FRONT_ROLL_DAYS	8	Roll front kontraktu (#1189, ADR-0039): kontrakt je front, dokud má do expirace VÍC než N dní (CME roll date = 8 d před expirací). Platí pro IBKR

Proměnná	Default	Význam
		pipeline, tasty streamer i IV rank. 0 = původní chování (nejbližší nepropadlý kontrakt). Discovery cache front kontrakt po rollu zahodí.
GEXLENS_WATCHLIST_POLL_CYCLES	5	Watchlist + runtime nastavení (strike_range_points) se čtou z DB každý k-tý cyklus
GEXLENS_OI_ARCHIVE_EXPIRIES	5	Ranní OI archiv pokrývá N nejbližších expirací (základ Δ OI vs. včera)
GEXLENS_SWEEP_NEXT_EXPIRY	true	Sekundární sweep následující expirace (positioning příští seance)
GEXLENS_NEXT_EXPIRY_SWEEP_EVERY	3	Kadence sekundárního sweepu (každá k-tá minuta)
GEXLENS_STRIKE_RANGE_POINTS	200	Výchozí denní obálka spot $\pm X$ (ADR-0002)
GEXLENS_STRIKE_RANGE_EXPAND_THRESHOLD	0.25	Rozšíření při přiblížení k okraji
GEXLENS_STRIKE_RANGE_MAX_POINTS	800	Strop šířky obálky ($\geq 2 \times$ base)
GEXLENS_BATCH_SIZE	80	Dávka rotačních subskripcí
GEXLENS_BATCH_TIMEOUT_S	4	Čekání na kompletní data kontraktu
GEXLENS_WINGS_SWEEP_EVERY	3	Křídla každý k-tý cyklus
GEXLENS_ATM_SWEEP_WIDTH	30	ATM $\pm N$ strikes každý cyklus
GEXLENS_REPAIR_MAX_ATTEMPTS	3	Retry repair fronty za sweep
GEXLENS_MARKET_DATA_LINES	100	Kapacita market data lines — tvrdý strop účtu je 100 (změřeno #609; původní odhad „ ≥ 150 “ z ADR-0001)

Proměnná	Default	Význam
		neplatil). <code>batch_size</code> nikdy nezvyšovat
<code>GEXLENS_CUMDELTA_SOURCE</code>	midpoint	Zdroj znaménka CumΔ (ADR-0032): <code>midpoint</code> = minutový test celý řetěz; <code>dxfeed</code> = tisky TimeAndSale se stranou od burzy, midpoint jen fallback per kontrakt a minutu. Přepnout až po srovnání řad a rozhodnutí
<code>GEXLENS_DATABASE_URL</code>	postgres localhost	V compose směřuje na službu <code>postgres</code>
<code>GEXLENS_DATA_DIR</code>	data	Kořen Parquet partic
<code>GEXLENS_RETENTION_DAYS</code>	90	Purge okno (ADR-0022, odchylka od R3). Nemaže se: <code>oi_eod</code> ani žádná <code>bars/</code> partice
<code>GEXLENS_KEEP_BARS_FOREVER</code>	true	Bary podkladu se z purge vyjímají — základ historických výpočtů (ADR-0028)
<code>GEXLENS_DISK_LIMIT_GB</code>	2	Alert při překročení
<code>GEXLENS_RETENTION_PURGE_TIME_UTC</code>	21:30	Čas nočního purge
<code>GEXLENS_API_BASE</code>	http://127.0.0.1:8000	Kam engine pushuje (v compose <code>http://api:8000</code>)

| `GEXLENS_PG_PASSWORD` | — | **Povinné** — compose bez něj nenastartuje (generuje `scripts/init-secrets.ps1`) | | `GEXLENS_API_TOKEN` | — | **Povinné** — sdílené tajemství `/internal/*` a `/backup/postgres` | | `GEXLENS_BIND_ADDR` | 127.0.0.1 | Na jaké adrese publikují porty (server: ponechat loopback + reverse proxy) | | `GEXLENS_ALLOWED_ORIGINS` | — | CORS whitelist API | | `GEXLENS_NEWS_API_TOKEN` | — | Token news-engine → API push | | `GEXLENS_TASTY_ENABLED` | true | **Trvalá** tastytrade větev (#613, #763): session, DXLink stream, chain mapa, křížová kontrola (#517 A), oba fallbacky (#614), OI fill (#664). Bez tajemství se stejně nespustí, proto default `true` | | `GEXLENS_TASTY_COMPARISON_WRITE` | true | **Dočasný** zápis porovnávacích řádků do `feed_comparison`

(#613). Vypnout po vyhodnocení M7 fáze 2 — tally pro detektor a fallbacky běží dál, takže se tím NEztrácí odolnost proti výpadku IBKR. Surová historie do 22. 8. je zhuštěná do

```
feed_comparison_daily (#965, kap. 3) | | GEXLENS_TASTY_MAX_ENTRIES /
```

GEXLENS_TASTY_ADHOC_RESERVE_ENTRIES | 25 000 / 2 000 | **Rozpočet DXLink subskripce** (#982, ADR-0027): server počítá položky `symbol × event` na spojení a strop je 25 000 (změřeno sondou `tasty_probe.py sizecap`, 2. 9.) — produkce 6 236 symbolů × 4 eventy = 24 944, takže ad-hoc pohled přetekl a odmítnuté symboly tiše mlčely. `tasty/budget.py` odebírá **per účel jen eventy, které se čtou** (wide Quote+Summary — jde o Ol; extended a ad-hoc Quote+Greeks+Summary; řetěz a podklad vše), produkce ≈ 18 400 položek; wide/extended smí jen po `strop - rezerva`.

Deterministický ořez při přetečení: extended od nejvzdálenější expirace/striku, pak wide od okraje — řetěz/podklad/ad-hoc nikdy. `/status.tasty_budget` nese využití, ořez a `size_exceeded` (server nahlásil `subscription size is too big`; heal se na to nespouští) | | GEXLENS_TASTY_SHADOW | — |

ZASTARALÉ (#763), nechávej nenastavené. Hlídalo obojí naráz, takže „vypínám doběhnuté měření“ tiše bralo i fallbacky. Když je nastaven, řídí `GEXLENS_TASTY_ENABLED` a engine to při startu ohlásí varováním | | GEXLENS_TASTY_CLIENT_SECRET / _REFRESH_TOKEN | — | OAuth2 grant **výhradně scope read** (ADR-0025); dev grant patří do `.env.dev` pod standardními názvy. Obsah `.env` se nikdy nevypisuje do konzole | | GEXLENS_CROSSCHECK_ENABLED | true | Křížová kontrola IBKR × tasty (#517 fáze A) — pasivní, bez requestů a linek navíc. Bez zapnuté tasty větve se tiše nezapne | |

GEXLENS_CROSSCHECK_SHARE_THRESHOLD / _MINUTES / _COOLDOWN_MINUTES | 0.70 / 3 / 15 | Prahy **měřené** na shadow historii, ne odhadnuté — viz níže. `_MINUTES` pod 3 = falešný poplach každou třetí minutu | | GEXLENS_CROSSCHECK_CHANGE_THRESHOLD | 0.30 | Rozlišovač „tichý trh × mrtvá záloha“ (#764): podíl kontraktů s **měnicími se** IBKR hodnotami, od kterého je trh živý a mlčící tasty porucha → alert `feed_backup_dead`. Měřeno nad 4 833 min (13.–19. 8.): pauza CME má medián změn ~0, aktivní trh $p_5 \approx 0,38\text{--}0,42$; s prahem 0,30 vyšlo **0 planých poplachů** (bez rozlišovače 6 epizod, všechny v pauze CME 21–22 UTC) | | GEXLENS_DISK_FREE_WARN_GB / _CRIT_GB /

GEXLENS_DB_SIZE_ALERT_GB | 15 / 5 / 4 | Dohled nad volným místem (#773): alert `disk_space` do zvonečku (varování/kriticky) + výpis největších tabulek. Měří se SKUTEČNÉ volné místo disku s datovým adresářem (bind mount ukazuje čísla hostitele) a velikost PG přes `pg_database_size`. Obsazení hostitelského disku WSL vhd_x (u nás `D:\Programy\Docker\DockerDesktopWSL`, PG volume uvnitř) z kontejneru změřit nejde — protože ale vhd_x leží na témže disku jako data, jeho růst měřené volné místo ukusuje přímo; práh na velikost DB je časná výstraha na tahouna růstu. Vhd_x se po úklidu sám nezmenší — kompaktace vyžaduje zastavený Docker (`ws1 --shutdown + Optimize-VHD /diskpart`). Stejná čísla plní patičku UI (`Disk X / Y`), která do té doby ukazovala `- / -` | | GEXLENS_PROBE_ENABLED | true | Aktivní IBKR sonda (#517 fáze B): na alert `ibkr_suspect` jednorázový snapshot front future → rozliší výpadek farmy („čekej“) od potichu mrtvých subskripcí (cílená obnova bez reconnectu). Neběží periodicky; vlastní cooldown 10 min a použít se jen s rezervou ≥ 2 market data lines | | GEXLENS_TASTY_SPOT_FALLBACK | true | Cena podkladu z `tastytrade`, když IBKR přestane posílat ticky (#614 fáze 2a) | | GEXLENS_TASTY_SPOT_STALE_AFTER_S / _RECOVER_AFTER_S / _MAX_AGE_S | 30 / 60 / 30 | Hystereze spotu: kdy převzít, kdy vrátit, max stáří

tasty kotace. Návrat je delší než převzetí schválně — kmitání zdroje stojí víc než o půl minuty pozdější návrat || `GEXLENS_TASTY_CHAIN_FALLBACK` | true | Fallback **celého opčního řetězu** (#614 fáze 2b). Spouští ho verdikt křížové kontroly, takže dědí její měřené prahy ||

`GEXLENS_TASTY_CHAIN_RECOVER_MINUTES` | 5 | Kolik čistých minut v řadě vrátí řetěz na IBKR. Delší než zapínací série (`_CROSSCHECK_MINUTES`): přepnutí překreslí celý profil || `GEXLENS_TASTY_CHAIN_MAX_AGE_S` | 120 | Max stárí tasty hodnoty, aby kontrakt vstoupil do fallbackového řetězu ||

`GEXLENS_STARTUP_CONNECT_WAIT_S` | 60 | Jak dlouho se při startu čeká na IBKR, než engine rozjede zbytek i bez něj (#756). **Není to timeout spojení** — supervisor se pokouší dál. 0 = nečekat ||

`GEXLENS_TASTY_OI_FILL` | true | Díry denního OI archivu doplní tasty `Summary` (#664) — typicky ODTE ráno, než CME publikuje || `GEXLENS_NEWS_LLM_ENABLED` | false | LLM klasifikace zpráv (Gemini). **Zakonzervováno** (#740 fáze 0): v `news_weights` neprošla Wilson gate ani jednou (0/20 řádků, hit rate 0,484 proti 0,516 u pravidel). Od ADR-0036 (#1150) váha z Wilson LB nikdy není 0 (rozsah 0,25–2,0, mince = 1,0) a event se váží řádkem svého predictorů (`sentiment_source`) ||

`GEXLENS_NEWS_EXPLAIN_ENABLED` | false | **Vysvětlení zprávy** (#1126 3d, v1.18): tlačítko „Vysvětlit“ u karty v News → `POST /news/{id}/explain` → Gemini free tier přes REST (klíč `GEXLENS_NEWS_GEMINI_API_KEY` výše, api kontejner přes `env_file` ; bez klíče 503, cache jede dál). Jiný účel než klasifikace: porozumění, ne predikce — do SentIndexu, vah ani signálů nic neteče (R4). Odpovědi navždy v PG `news_explanations` (event_id, model, text, tokeny, created_at) ||

`GEXLENS_NEWS_EXPLAIN_MODEL` | gemini-3.8-flash,gemini-3.6-flash,gemini-3.5-flash | Řetěz modelů oddělený čárkou — při 5xx „high demand“ se hned zkusí další (free tier je přetěžovaný, 15. 9.: 3.8 200/503/503, 3.7 503). **Pinovat konkrétní verze**, ne alias `*-latest` (#738); `thinkingLevel`: low (`minimal` odmítá 400). Jedno vysvětlení ≈ 600 tokenů; uložený řádek nese model, který odpověděl || `GEXLENS_NEWS_EXPLAIN_DAILY_TOKENS` | 300000 | Denní strop prompt+output tokenů za UTC den, po překročení 429 (0 = bez stropu); free tier má navíc vlastní denní kvótu requestů (429 od Google se hlásí stejně). Spotřeba je v logu api (`Vysvětlení zprávy N: model, in+out tokenů (dnes X/strop)`) || `GEXLENS_PUSH_TELEGRAM_TOKEN` | — | **Push na Telegram** (#1175, v1.18): přihlašovací řetězec bota z @BotFather; api kontejner ho dostane přes `env_file` . Bez něj (nebo bez chat id) se nic neposílá, `GET /push/status` hlásí `configured: false` || `GEXLENS_PUSH_TELEGRAM_CHAT_ID` | — | Id soukromého chatu s botem (napiš botovi /start, id je v odpovědi metody `getUpdates` Bot API, pole `chat.id`) || `GEXLENS_PUSH_QUIET_HOURS` | 23:00-06:00 | Tiché hodiny v Europe/Prague (`HH:MM-HH:MM` , prázdné = žádné); provozní kategorie jde i v nich || `GEXLENS_PUSH_DAILY_CAP` | 200 | Denní strop odeslaných zpráv (0 = bez stropu); dedup per (kind, symbol, začátek textu) 10 min ||

`GEXLENS_PUSH_SETUP_MIN_CONFIDENCE` | 0 | Minimální confidence (0–1) vzniklého setupu pro push; engine ji od #1175 posílá v alertu jako číslo || (*serverová nastavení*)

`push_telegram_setup/ops/news/info` | true/true/true/false | Přepínače kategorií (Settings → Notifikace), čte je api při každém alertu. Zdroj alertů = kanál `alerts` `LiveHub` (posluchač `TelegramPush`), takže engine, news-engine i provozní alerty api jdou jednou cestou; odeslání běží v daemon vlákně, 3 pokusy, 4xx = chyba konfigurace bez opakování; stav v `GET /push/status` ||

`GEXLENS_SCENARIO_AUTO_MINUTES_BEFORE_OPEN` | 15 | **Automatický scénář dne** (#1173 A): kolik minut

před US openem (9:30 ET) engine per symbol sestaví scénář z verdiktu dne (`compute/dayverdict` , port frontendového hlasování, `VERDICT_RULES_VERSION` musí sedět s `instrument/daysummary.ts`); vstupy čte z API stejnými endpointy jako Briefing (`/candles` , `/bars` , `/instruments/{sym}/days` , `/oidelta` , `/sentiment/state` , `/news/upcoming`), levels a tendence z runtime. Jeden pokus per seance v okně [open – N, open); `source='auto'` , `rationale` = hlasy + chybějící vstupy; verdikt `none/wait_news` = bez scénáře (log). Snímek doplní frontend přes `PUT /scenarios/{id}/image` (jednou, jinak 409). 0 = vypnuto | | (scénáře dne, #1173) | — | PG tabulka `scenarios` (řádek per scénář: symbol, den a čas vzniku, termín + `deadline_ts` = settle termínu, vstup, cíle, cesta, `annotation_id` , `image_path` , `image_bytes` , `evaluated_at` , `result`), snímky `data/scenarios/{sym}/{den}/{id}.png` (max 4 MB, jen PNG). API: `POST /scenarios` (jen dopředu — termín ≥ aktuální seance, do 60 dní; `created_at` razítkuje server), `GET /scenarios?symbol=` , `GET /scenarios/{id}/image` , `GET /scenarios/stats` (`preliminary` do `n < 30`), `GET /scenarios/disk` (součet z DB, ne rglob), `DELETE /scenarios/images?older_than_days=N` (jen PNG, řádky zůstávají), `DELETE /scenarios/{id}` . Engine `ScenarioCollector` per symbol jednou po settle (+15 min): scénáře po termínu bez výsledku → bary `derived/{sym}/bars/` od `created_at` do `deadline_ts` , EM z `em_respect` seance vzniku → `result` + alert `scenario_result` (kategorie push „Setupy a scénáře“). Nad 1 GB snímků alert `scenario_disk` (hranově, kategorie provoz); žádné automatické mazání | | `GEXLENS_NEWS_ALPACA_KEY_ID` / `_SECRET` | — | Alpaca News API — živý stream i historický backfill (#743, #744). Stačí paper účet a **Trading API**, ne Broker API | | `GEXLENS_NEWS_REDDIT_RSS_ENABLED` / `_INTERVAL_S` / `_FEED_DELAY_S` | `true` / 300 / 15 | Reddit nativní RSS (#578). Reddit limituje **anonymní přístup per IP napříč subreddity** (#941, měřeno 29. 8.: 5 s → 429, 15 s → 200, dva feedy za sebou i tak 429 — potřebuje ~150 s klidu). Kolektor proto jede **round robin — jeden subreddit za cyklus** (každý à 10 min, pro crowd sentiment stačí), s jedním retry při 429; chyby feedů se logují se status kódem |

Od #696 jde do kontejnerů celý `.env` (`env_file`), ne ruční výčet — každý klíč z `.env.example` po `docker compose up -d <služba>` skutečně platí. `environment:` v compose nese jen odvozené hodnoty (`DATABASE_URL`, adresy služeb, `/app/data`) a ty mají přednost. **Dev stack navíc čte volitelný** `.env.dev` (přepisy jen pro dev: vlastní tasy grant, symboly...; viz `.env.dev.example`). Pozn.: změna PG hesla v `.env.dev` vyžaduje reseed dev volume.

Frontend build-time: `VITE_API_BASE` (nginx build arg, default `http://127.0.0.1:8000`).

5. Engine — datová pipeline

Minutový cyklus (`runtime.EngineRuntime.run_cycle`):

1. **Sweep** — `SubscriptionScheduler` projede řetězec v dávkách (ATM±30 každý cyklus, křídla každý 3.), nekompletní kontrakty přes repair frontu, výsledek do in-memory cache.

2. **Snapshot** — cache → `SnapshotRow` (OI z ranního archivu) → atomický zápis Parquet.
3. **Výpočty** — GEX per strike (naivní dealer model, vyměnitelná strategie) → levels (flip interpolovaně, walls, centroid) → zápis do `derived/levels`.
4. **Cum Δ** — bar větev ($\Delta \text{Vol} \times \text{midpoint test} \times \Delta \times M$) pro celý řetěz; trade větev z `dxFeed` `TimeAndSale` pro celý sbíraný řetěz přijde s #615 fází 3 (ADR-0032 — IBKR hot zóna z původního návrhu se nikdy nenapojila, CumΔ do té doby je 100 % midpoint). `close_minute` → `derived/flow`.
5. **Bary podkladu** — 5s `reqRealTimeBars` agregované na 1min → `derived/bars`.
6. **Push do API** — `/internal/status` + kanály `levels.*`, `flow.*`, `price.*`.

Multi-instrument orchestrátor (ADR-0003)

`__main__` řídí **pipeline per podklad** (`instruments.InstrumentPipeline`): cílová sada = `GEXLENS_SYMBOLS` U watchlist z DB — změny chodí okamžitě přes PostgreSQL `LISTEN/NOTIFY` (kanál `gexlens_watchlist`, #207: API po zápisu notifikuje, orchestrátor se probudí ze sleep a nový symbol startuje do sekund; svíčky dne doplní backfill z #221), poll à `WATCHLIST_POLL_CYCLES` zůstává jako fallback pro backendy bez NOTIFY. Probuzení uprostřed minuty spustí plný cyklus jen pro nové pipeline — běžící by duplikovaly zápisy. Sweepy instrumentů běží **sekvenčně** — špička market data lines je vždy jedna dávka. Multiplikátor a burza se čtou z contract details. Ne-futures symbol → alert `instrument_error` + cooldown 30 cyklů. Pád cyklu jednoho instrumentu neshodí ostatní; status se agreguje (součty Greeks/repair, pole `symbols`).

Každá pipeline navíc drží **sekundární runtime následující expirace** (`secondary=True`): sweep v kadenci `NEXT_EXPIRY_SWEEP_EVERY`, zapisuje jen snapshots + levels své expirace (flow/bary patří výhradně aktivnímu řetězu — soubory jsou per symbol).

Další joby: **OI archiv** při startu + retry à 30 min dokud den nemá data (alert `oi_missing`); pokrývá `OI_ARCHIVE_EXPIRIES` nejbližších expirací — základ ΔOI vs. včera. **POZOR: OI se čte přes generic tick 101 i pro FOP** (tick 588 na FOP nedodává nikdy — ADR-0001 v3; hodnota se čte podle strany kontraktu, opačná strana je validní 0.0). **Auto-rozšíření obálky strikes** (grow-only, capped → alert) + runtime změna `strike_range_points` ze Settings UI (překlopí pipeline). **OI zdi** (#851): `compute/oiwalls.py` počítá maximum OI per strana nad širokým archivem (ne nad snapshoty omezenými obálkou) s cache na `captured_ts` — archiv se přes dopoledne dopisuje, klíč jen na den by zamrzl jako Max Pain (#826); vlastní řada `oiwalls/`, `LEVELS_SCHEMA` se nerozšiřuje (ADR-0008).

Nastavení připojení ze Settings UI (#446, #950, #992): orchestrátor čte watchlist + runtime nastavení z DB každý `WATCHLIST_POLL_CYCLES`-tý cyklus nebo po `LISTEN/NOTIFY`; od #992 posílá NOTIFY i `PUT /settings/{key}` (stejný kanál `gexlens_watchlist` — po probuzení se čte obojí jedním průchodem) a **bez spojení k IBKR se DB čte každý cyklus**

(`runtime_settings.should_poll_settings`), takže změna portu platí do sekund i v reconnect smyčce (dřív až za ≤ 5 min). Hodnota uložená v DB **přebíjí** `.env` — je to záměr #446, ale po `docker compose up -d engine` s přepsaným `.env` to jinak nešlo poznat (2. 9.: connect na 4001 a o sekundu později skok zpět na 7496); engine to od #992 při prvním cyklu hlásí `WARNING` em s návodem, co změnit. **Ruční přepojení** (#950): `POST /engine/reconnect {target: ibkr|tasty|both}` zapíše serverem generované razítko `reconnect_request_*` do `settings` (klíče schválně nejsou ve `WRITABLE_SETTINGS`, přes `PUT /settings` je podvrhnout nejde); engine reaguje na **změnu** razítka (`pending_reconnects`), výchozí stav si načte `seed_reconnects` jednou před hlavní smyčkou — chybějící klíč se pamatuje jako `None`, takže první požadavek po startu neshoří (#957). IBKR: `ib.disconnect()` + supervisor; tasty: `DxLinkStream.force_reconnect()` (zavře socket, standardní `run` smyčka udělá reconnect i resubskripci). Přepojení = 1–2 min díra ve sběru, UI si vyžádá potvrzení. **Denní roll expirace**: vypršelá pipeline se zastaví a další cyklus založí novou s čerstvou discovery (bezobslužný přechod přes víkend). **Noční retention purge** po `RETENTION_PURGE_TIME_UTC`.

Bary podkladu (#221): **Backfill 1min barů** při startu pipeline (aktuální den + retention okno, `reqHistoricalData` pod `pacing guardem`, `upsert` podle `ts_min` — živý stream a backfill se nedubluje; od #1055 (v1.6) nese doplněný bar `source = ibkr_hist` a **změřenou minutu nepřepíše** — přednost původu `bar_source_rank : měřený > ibkr_hist > tasty_candle`, živý zápis historickou hodnotu naopak nahradí vždy). **Hlídní tiché ztráty barů** (`BarsStallDetector`): když $\geq \text{BARS_STALL_ALERT_MINUTES}$ (default 3) nedorazí žádný 5s bar, ale spot se hýbe, odejde alert `bars_stalled` (typicky mrtvé TWS farmy po noční přestávce — pomáhá restart TWS); po návratu streamu alert `bars_recovered` + automatický re-backfill dnešního dne doplní díru. Bez pohybu spotu (zavřený trh) se nehlásí nic. **Rekonstrukce děr z dxFeed Candle** (#617, v1.3): jednou po startu pipeline (`_candle_gap_backfill`, jen s běžící tasty větví) se pro aktuální seanci spočítají minuty, které IBKR historical nedodal, a doplní se z `dxFeed Candle` (historie od `fromTime`, bez `pacing limitu`) přes vlastní krátké spojení mimo živou datovou cestu — sdílený handshake `tasty/dxlink.py . backfill_gaps` výsledek ještě jednou filtruje na chybějící minuty, takže měřená minuta se nemá jak přepsat; selhání se jen zaloguje. ADR-0024 platí dál pro opční vrstvu (Greeks zpětně neexistují). Past z ADR-0027: streamer symbol se nesestavuje (`/ESU6:XCME` s hlubokým `fromTime` vrací rok 2016), bere se hotový z chain endpointu. UI doplněné minuty hlásí bannerem (sbírá se ze všech barů dne, ne jen z těch na ose snapshotů — večerní minuty Globexu na osu opcí nepadnou, #974). **Hlídky Greeks po settle** (#959): `greeks_watch_applies(expiry, now)` vypne `greeks_stalled` pro expirující řadu po jejím settle (`compute/settle.py`, DST-korektně) a detektor se nekrmí — vypořádaný řetěz se přestane kotovat legitimně (sekundární řada měla v téže vteřině plný počet) a pipeline nad ním běží až do půlnoci, kdy `expiry_expired` překlápí podle kalendářního dne.

Odolnost: **reconnect nesmí umlknout** (#770): `_supervise()` je odolná smyčka nad `_try_connect()` — výjimka v iteraci (padlý odběratel stavu, selhaná resubskripce) se zaloguje a jede se dál, selhaná resubskripce jde rovnou na reconnect; **watchdog běží záměrně mimo**

supervisora a křísí mrtvou smyčku (čítač `ConnectionManager.supervisor_restarts` — nenulová hodnota je nález, do logu jde jako vzkříšení supervisora), po `RECONNECT_STALL_ALERT_S` hlásí `connection_stall` (18. 8. byl engine 8 h offline bez jediného řádku). ConnectionManager watchdog (heartbeat 30/15 s + exponenciální reconnect + plná resubskripce — **vč. spot tickeru a realtime barů podkladu** přes `on_resubscribe`), spot fallback last → marketPrice → close (start i o víkendu), discovery s timeoutem a retry (sec-def farm výpadky), výjimka v cyklu nikdy neshodí smyčku, pacing guard historical requestů (≤60/10 min, dedup, prioritá).

6. Datové formáty a persistence

Parquet (`GEXLENS_DATA_DIR` ; **retence** — **ADR-0029**)

Od ADR-0029 (v1.2) se `snapshots/` a `derived/` **nemazou nikdy** (`GEXLENS_KEEP_LEARNING_DATA_FOREVER=true` , default) — jsou to nenahraditelná učící data samoučící smyčky (#794); IBKR historii řetězce zpětně nedá. Noční purge (`RETENTION_DAYS` , ADR-0022: 90 dní) tak reálně maže jen `ticks/` . Objem keep-forever režimu ≈ 6 GB/rok (ES+NQ); `GEXLENS_DISK_LIMIT_GB` (default 20) je alert na revizi, volné místo hlídá #773.

Partice	Schéma
<code>snapshots/{sym}/{expiry}/{YYYY-MM-DD}.parquet</code>	ts_min, strike, right, bid, ask, last, volume, iv, delta, gamma, theta, vega, oi, stale_age
<code>ticks/{sym}/{YYYY-MM-DD}.parquet</code>	ts, conld, price, size, side
<code>derived/{sym}/{expiry}/levels/{date}.parquet</code>	ts_min, flip, call_wall, put_wall, centroid, total_gex
<code>derived/{sym}/flow/{date}.parquet</code>	ts_min, flow_delta, cum_delta, futures_cvd*, source (v1.4, ADR-0032: <code>midpoint</code> / <code>dxfeed</code> ; NULL = partice před #615 fázi 3 = midpoint), od #1071 (v1.6) pokrytí trade větví za minutu: <code>printed_volume</code> , <code>unknown_volume</code> , <code>structured_volume</code> , <code>fallback_volume</code> , <code>dropped_no_delta</code> (NULL = trade větev neběžela nebo partice před #1071; měří se i v režimu midpoint — říká, co by dxFeed pokryl; <code>/status.cumdelta_coverage</code>

Partice	Schéma
	je totéž od startu enginu). Čte scripts/compare_cumdelta_sources.py (sloupce „pokrytí tisky“ a „fallback RTH“)
derived/{sym}/{expiry}/printvol/{date}.parquet	Podíl objemu mimo tisk (#1007, v1.4): per kontrakt a bar volume_delta , printed (Σ tisků TimeAndSale od minulého baru), structured (zbytek bez tisku — nohy spreadů, bloky). NULL = trade větev pro instrument neběžela (tasty odpojené, symbol bez univerza); nula by lhala „100 % struktura“. Jen řádky s přírůstkem; push printvol.{sym}.{exp} , sekce printvol v /replay
derived/{sym}/bars/{date}.parquet	ts_min, open, high, low, close, volume, source — z purge vyňaté, drží se navždy (GEXLENS_KEEP_BARS_FOREVER=true); ES i NQ mají ~2 roky historie od 2024-07- 28 (backfill scripts/backfill_bars.py). source (#617): NULL = partice před #617, ibkr = živá cesta, ibkr_hist = doplněno z IBKR historical (#1055, v1.6 — platná cena, ale engine v tu minutu neměřil; do 8. 9. 2026 se zapisovalo jako ibkr), tasty_candle = rekonstruováno z dxFeed; pyarrow čte starší šestisloupcové soubory novým schématem s NULL (ověřeno). Frontend z něj staví historii přes hranici dne (#788, GET /bars?date= den po dni, 404 = víkend, 5 děr v řadě = konec archivu). Partice = UTC den baru (#1002, v1.3): do 3. 9. 2026 engine zapisoval půlnoční bar 23:59 i rekonstruovaný blok 22:00–23:59 dne D–1 do partice sousedního dne → minuta dvakrát, objem dvojnásobný v oknech přes půlnoc; staré partice opraví scripts/fix_bar_partitions.py --apply

Partice	Schéma
	(spouštět při zastaveném enginu; bez <code>--apply</code> jen vypíše plán), API a news-engine navíc deduplikují obranně
<code>derived/{sym}/{expiry}/oiwalls/{date}.parquet</code>	OI zdi (#851): <code>oi_call_wall / oi_put_wall + share</code> (podíl na OI strany; frontend pod 0,2 nekreslí)
<code>derived/{sym}/features/{date}.parquet</code>	Minutový feature log (#796): vstupní vektor setup detektoru + ATR + band metriky — trénovací matice smyčky #794
<code>trades/{sym}/{YYYY-MM-DD}.parquet</code>	Surové opční TimeAndSale printy z dxFeed (#795): <code>ts, streamer_symbol, price, size, aggressorSide, spread_leg, eth</code> . Mimo retenci; podklad budoucí klasifikace agresora (#615). Flag <code>GEXLENS_TASTY_TRADES_RECORD</code> (default true)
<code>derived/{sym}/{expiry}/netflow/{date}.parquet</code>	Kumulativní klasifikovaný net objem per strana (midpoint/Lee-Ready) — podklad FA odhadu OI a ranní kalibrace α . Píše aktivní i sekundární řetěz (#1182): aktivní ES/NQ je vždy 0DTE bez Δ OI do D+1, kalibrace proto bere netflow sekundáru (expirace po dni netflow); <code>fa_alpha_history.expiry</code> říká, ze kterého řetězu bod vznikl.
<code>derived/{sym}/{expiry}/oiest/{date}.parquet</code>	FA odhad OI ($\text{netflow} \times \alpha$, #232)
<code>derived/{sym}/{expiry}/gexprofile(fa)/... + gexfield(fa)/...</code>	Dyn profily/pole; <code>...fa</code> varianty nad FA odhadem
<code>derived/{sym}/{expiry}/charmprofile/..., vannaprofile/... (+ ...field)</code>	Dyn Charm/Vanna plochy (#204)
<code>derived/{sym}/{expiry}/greekssource/{date}.parquet</code>	Zdroj greeks per minutu (model/computed, #547)
<code>derived/{sym}/{expiry}/oimissing/{date}.parquet</code>	Striky bez OI (šrafura, #465)

Partice	Schéma
<code>derived/{sym}/catchup/{date}.parquet</code>	Příznak dohánění po startu (#518)
<code>derived/{sym}/gexforward/{date}.parquet</code>	Forward GEX (#519): bloky per budoucí obchodní den (day, grid, values, dropped_expiries, dropped_share, iv_fallback_share); jen poslední stav, přepočet po OI archivu
<code>derived/sentiment/{SYM}/{date}.parquet</code>	1min řada SentIndexu per symbol (ADR-0026; ploché soubory bez symbolu = ES legacy)

Zápis je **atomický** (temp + rename) — po pádu procesu nikdy nezůstane částečný soubor; osiřelé `.tmp` se uklízí při dalším zápisu. Writer po restartu navazuje na rozepsaný den.

PostgreSQL

Tabulka	Účel
<code>oi_eod(symbol, expiry, trading_class, strike, right, date, oi, iv, delta, gamma, theta, vega, close_prem, und_price)</code>	Věčný denní snímek řetězce — od #519 nese vedle OI i IV/greeks/závěrečnou prémii/ref. spot z ranního průchodu (NULL = model nedodal). Od #736 je v klíči <code>trading_class</code> (série se neslévají — E4C/EW4/EW...; <code>' '</code> = souhrn/legacy, historie čitelná dál; čtení Σ přes série dává konzumentům stejná čísla jako dřív, <code>values_for(trading_class=...)</code> pro kalendář #513). Žádná retence, žádné delete API
<code>gamma_cliff</code>	Denní odpad gammy po expiraci + metriky následující seance (#576, fáze měření): <code>next_range_atr</code> , <code>next_setups</code> a od #1115 <code>next_outside_share</code> = podíl minut následující Globex seance (do settle) s <code>band_depth ≤ 0</code> (třídy outside/no_zone, totéž pravidlo jako stínová brána #1060) z feature logu <code>derived/{sym}/features/</code> ; NULL = feature log seance chybí, doplňuje se při dalším běhu
<code>feed_comparison</code>	Shadow porovnání IBKR × tastytrade per (minuta, kontrakt, pole) — jen po dobu sběru M7 fáze 1 (#613). Historie do 22. 8. smazána 1. 9. (#965)
<code>feed_comparison_daily</code>	Zhuštěné souhrny shadow sběru (#965): denní řádky + celkové řádky (<code>session_date IS NULL</code>) per symbol × pole s <code>n</code> , mediánem a

Tabulka	Účel
	p95
sentiment_daily , sentiment_waves , sentiment_episodes , news_* , signals , signal_outcomes , track_record	SentimentLens (per symbol od ADR-0026). sentiment_episodes (#565, ADR-0037) = korekční epizody pokus/negace, plně derivované — WavesJob je přepočítává full-replace z close_z ; sloupec params_version nese verzi prahu D / horizontu H (v1 = placeholder 1 σ / 10 dní), změna parametrů = nová verze v gexlens_engine.compute.sentwaves a přepočet při příštím běhu jobu. Přeměření: uv run python scripts/measure_sentiment_episodes.py --era both --out ... (prod DB jen čtení; URL z GEXLENS_PG_PASSWORD , port 55432). news_reactions je od #998 (ADR-0031) jeden řádek per event × symbol se sloupci per okno (ret_<w> , range_<w> , vol_z_<w> jen minutová okna, cont_<w>) a per fázi (deferred_* , regime_* , computed_at_*) — 268 → ~56 MB, bez retence (učicí data). Starý tvar (řádek per okno) news-engine při startu odmítne s odkazem na scripts/migrate_news_reactions_wide.py (jednorázově, po záloze PG; --dry-run napřed); stará tabulka zůstane jako news_reactions_legacy_<datum> a maže se ručně až po ověření provozu
setups	Setupy vč. context JSON (od #575 nese band_sharpness/band_sharpness_pct/band_depth; od #952 i band_metrics_version = 2 — hloubka pásma nad Major se mapuje na (1, 2] místo saturace na +1, v1 a v2 se nesmí sdružovat; po rebuildu spustit scripts/backfill_band_metrics.py , idempotentní podle verze) a mechanics_version (v5 od #859: setupy z doby zamrzlého Max Painu (#826) se nehodnotí — nemažou se, jen se verzí vyřazují ze statistik)
paper_accounts , paper_events , paper_orders	Paper účet (#1187 fáze 1, ADR-0040): účet 1 (start 50 000 \$ v jednotkách plného kontraktu, halted = kill switch), události (vklad/výběr/poznámka), ordery (side, qty, typ, entry/stop/cíl, status working/open/closed/cancelled , fill/exit ceny a čas, exit_reason stop/target/manual/settle/kill, pnl_usd , fees_usd , r_multiple , risk_usd , context JSON, journal_entry_id). Fily dělá engine per symbol po cyklu pipeline (gexlens_engine.paper.PaperBroker) proti 1min barům; uzavřený obchod zapíše do journal_entries / journal_trades (typ obchod , tag paper).

Tabulka	Účel
setup_params	Verzované prahy šablon setupů (#794 fáze 2, ADR-0033): append-only, poslední řádek platí; created_by (engine seed / ui / script), povinná note , mechanics_version , params JSON. Engine při prvním startu založí seed z .env + defaultů (od té chvíle store přebíjí .env klíče GEXLENS_SETUP_*), novou verzi přečte po NOTIFY nebo v k-tém cyklu. Setupy nesou params_version (NULL = před store).
adhoc_view	Most UI → engine pro ad-hoc pohled (#521 C), viz kap. 12
watchlist , alerts , annotations , settings	CRUD přes API

7. API reference

Interaktivní dokumentace: <http://127.0.0.1:8000/docs> (OpenAPI).

REST

Endpoint	Popis
GET /health , GET /status	Liveness; agregovaný stav pipeline (lines_utilization je od #630 měřená špička). Pole chain_source / spot_source nesou aktivní zdroj dat (#614), feed_crosscheck* verdikt křížové kontroly (#517 A), connection_offline_for_s délku výpadku IBKR (#770), tasty_rate_limited / tasty_heals / tasty_budget stav DXLink subskripce (#863, #936, #982), cumdelta_source + cumdelta_coverage[symbol] (v1.4, ADR-0032: printed_volume , unknown_volume , structured_volume , fallback_volume , dropped_no_delta , printed_share). Chybějící klíč znamená „neměří se“, ne „je to v pořádku“ — od #756 chodí status i bez jediné pipeline a connection nese skutečný stav spojení

Endpoint	Popis
POST /engine/reconnect { "target": "ibkr" "tasty" "both" }	Ruční přepojení (#950): zapíše razítko do settings , engine ho vyřídí při nejbližším pollu (kap. 5). Bez tokenu jako /settings — nová třída expozice nevzniká, kdo dosáhne na port, může spojení rozbít už přes PUT /settings/ibkr_port
GET /gexforward/{symbol}	Forward GEX bloky per budoucí den (#519)
GET /bars/{symbol}?date=	Lehké 1min OHLCV bary seance (#674/#678) — bez /replay balíku
GET /oidelta/{symbol}/{expiry}	ΔOI posledních dvou archivovaných dnů + top movers (#674)
GET /journal , POST/PATCH/DELETE /journal/*	Deník tradera (#673, fáze A)
GET /setups/params , POST /setups/params {params, note, created_by?}	Parameter store setupů (ADR-0033): platná verze + historie + defaulty; POST založí novou verzi (jen změněné klíče, zbytek defaulty; neznámý klíč/typ = 422, bez note = 422) a probudí engine NOTIFY. Autonomie stupeň 1: zapisuje člověk, ne smyčka.
— risk parametry (#1185)	Součást téže verze parametrů: account_equity_usd (50000), risk_pct (1), risk_max_pct (2), fee_per_contract_usd (10), daily_brake_r (3), weekly_brake_r (6), max_template_stops_per_day (2), template_gate_enabled (true), template_gate_min_samples (30), template_gate_days (60). Engine u každého setupu zapíše do context : risk_rules_version , contracts , risk_budget_usd , max_loss_usd , fee_usd , affordable , tradeable , trade_block (stop_over_budget / stop_over_cap / daily_brake / weekly_brake / template_stops / gate), template_gate (+ _n , _lb), realized_day_r , realized_week_r . Brzdy čtou uzavřené setupy napříč symboly (tradeable = true) od pondělní seance; brána šablon setupy se stopem v rozpočtu za template_gate_days seancí (starší řádky bez kontextu se dopočítají z entry/stop a hodnoty bodu). Alert risk_brake (kategorie push setup), setup alert nese

Endpoint	Popis
	<code>tradeable</code> — stín do pushe nejde. UI: Settings → Risk management (POST téže cesty).
<code>GET /gammacliff/{symbol}</code>	Dnešní odpad gammy + historie útesů (#576)
<code>GET /coach/review?date&symbol</code> , <code>GET /coach/weekly?date&symbol</code>	Kouč v1 (#933, <code>compute/coach.py</code>): obchody deníku typu obchod (ruční i paper) s příznaky (<code>no_stop</code> , <code>after_brake</code> , <code>revenge</code> , <code>big_loss</code> , <code>no_setup</code> , <code>low_rr</code> , <code>early_exit</code> — z barů po výstupu, <code>overtrading</code>), cenou v R, skóre disciplíny, shrnutím; týdenní report po–pá s cenou příznaků, rozpadem po hodinách UTC a 1–3 pravidly. Čte <code>journal_between</code> + <code>bars_session</code> ; nic nezapisuje.
<code>GET /paper/account</code> , <code>GET /paper/orders?symbol&status&limit</code> , <code>POST /paper/orders</code> , <code>DELETE /paper/orders/{id}</code> , <code>POST /paper/kill</code> , <code>POST /paper/resume</code> , <code>POST /paper/events</code> , <code>GET /paper/events</code>	Paper účet (#1187, ADR-0040). <code>POST /paper/orders</code> { <code>symbol</code> , <code>side</code> , <code>qty</code> , <code>order_type</code> market/limit/stop, <code>entry_price</code> (u market referenční cena), <code>stop_price</code> , <code>target_price?</code> , <code>setup_key?</code> , <code>setup_id?</code> , <code>note?</code> , <code>context?</code> } → 201 <code>working</code> ; 409 { <code>block</code> , <code>reason</code> , <code>max_contracts?</code> } při <code>kill_switch</code> , <code>position_exists</code> (jedna pozice/order na symbol), <code>daily_brake</code> / <code>weekly_brake</code> (z realizovaných paper obchodů, prahy z parametrů setupů) a <code>stop_over_budget</code> / <code>stop_over_cap</code> ($\text{equity} \times \text{risk_pct}$, strop <code>risk_max_pct</code>); 422 u neplatných úrovní / neznámého symbolu (hodnoty bodu v <code>compute.paper.POINT_VALUES</code>). <code>DELETE</code> nastaví <code>close_requested</code> — engine zruší čekající / zavře pozici na open dalšího baru. <code>POST /paper/kill</code> zastaví účet a zavře vše; <code>POST /paper/resume</code> odblokuje. Alerty <code>kind=paper</code> (placed/filled/closed/cancelled/kill; push kategorie setup), WS <code>paper.{symbol}</code> .
<code>GET /calendar/expiry?date=</code>	Kalendář expirací (#1189): fáze kvartálního týdne (<code>normal</code> / <code>roll</code> / <code>opex_week</code> / <code>expiry_day</code> / <code>post_opex</code>), <code>quarterly_expiry</code> , <code>roll_date</code> , <code>soq_ts</code> (9:30 ET), <code>vix_expiry</code> , značky do grafu (<code>markers</code> : <code>roll</code> , <code>quarterly_expiry</code> , <code>monthly_opex</code> , <code>vix_expiry</code>). Čistá kalendářní matematika bez DB. Engine posílá alert <code>expiry_calendar</code> (roll date 9:30 ET, pondělí OPEX týdne 8:00 ET, 5 min po SOQ; push kategorie news). Kvartální expirace propadá v SOQ

Endpoint	Popis
	(<code>compute.settle.expiry_settle_ts</code>) — po 9:30 ET se pipeline překlopí na další expiraci.
GET <code>/fa/alpha</code>	Kalibrovaná α FA odhadu per symbol (#232)
GET <code>/gexplane/{...}</code>	Dyn Charm/Vanna plochy (#204)
GET <code>/sentiment/*?symbol=</code>	Sentiment per symbol (ADR-0026): index/daily/state/waves
GET <code>/instruments</code> , GET <code>/instruments/{sym}/expiries</code>	Dostupné symboly/expirace (ze storage)
GET <code>/instruments/{sym}/days</code>	Uložené dny s expirací per den (Daily pohled)
GET <code>/profile/{sym}/aggregate?date</code>	Σ profil: OI/volume sečtené přes všechny expirace dne per strike (registrováno PŘED <code>/profile/{sym}/{expiry}</code>)
GET <code>/snapshots/{sym}/{expiry}?date&mode&scale&norm&from&to&raw</code>	Heatmap matice jako Arrow IPC stream ; <code>raw=true</code> = surová partice
GET <code>/levels/{sym}/{expiry}?date</code>	Časová řada flip/walls/centroid
GET <code>/profile/{sym}/{expiry}?date&ts&variant&oi_weight&spot</code>	Strike profil k okamžiku
GET <code>/flow/{sym}?date</code>	Cum Δ + OptVol + Vol řady
GET <code>/replay/{sym}/{expiry}/{date}</code>	Kompletní denní balík (levels/flow/bars JSON + snapshoty base64 Arrow + <code>oi_prev</code> pro Δ OI vs. včera)
CRUD <code>/watchlist</code> , <code>/alerts</code> , <code>/annotations?symbol&date</code> , <code>/settings</code>	PostgreSQL persistence
POST <code>/internal/status</code> , POST <code>/internal/publish</code>	Ingest z engineu — vyžaduje hlavičku <code>X-GEXLens-Token</code> (#542). Od #949 tu API vyhodnocuje provozní alerty <code>AlertEngine.observe_connection</code> / <code>observe_disk</code> (výpadek spojení s IBKR, obsazení disku přes limit) — ze snímku statusu, ne z těla requestu (engine posílá jen změněné klíče); obě hlášky jsou hranové. Do té doby byl <code>AlertEngine</code> mrtvý kód; pravidla <code>price_cross</code> / <code>cum_delta_jump</code> / <code>dominant_strike_change</code> odstraněna jako

Endpoint	Popis
	překonaná (<code>LevelProximityWatcher</code>), <code>POST /alerts</code> je přestává přijímat, CRUD <code>/alerts</code> zůstává
<code>PUT /settings/{key}</code>	Zápis nastavení + <code>pg_notify</code> na kanál watchlistu (#992) — engine se probudí do sekund
<code>GET /backup/postgres</code>	Stream <code>pg_dump -Fc</code> — vyžaduje <code>X-GEXLens-Token</code> (#542)

WebSocket `/ws/live`

Protokol: klient pošle `{"action":"subscribe","channels":["status","price.ES","levels.*"]}` (podpora trailing wildcard), server vrátí ack a pushuje `{"channel":..., "data":...}`. Backpressure: fronta 100 zpráv per klient, při zaplnění se zahazují nejstarší framy. Kanály: `status`, `price.{sym}`, `snapshot.{sym}.{expiry}`, `levels.*`, `flow.*`, `alerts`, `news`.

Handshake kontroluje hlavičku `origin` (#542): CORS se na WebSocket nevztahuje, takže bez téhle kontroly by živý positioning četla libovolná stránka otevřená v prohlížeči. Povolen je same-origin (Host stránky za nginx), localhost v libovolném portu a cokoli v `GEXLENS_ALLOWED_ORIGINS`. Klienti bez hlavičky `origin` (engine, curl) projdou. Stropy: 64 souběžných spojení, 256 kanálů na spojení.

8. Frontend

- **Heatmapa:** data → offscreen bitmap (překreslení jen při změně dat/módu), pan/zoom = GPU `drawImage` → 60 fps nezávisle na objemu; overlay canvas kreslí vektory (cena/svíčky, levels, walls, sessions, crosshair, anotace).
- **Replay:** `/replay` se stáhne jednou, `apache-arrow` dekoduje snapshoty, celý den se předpočítá v paměti (vč. profilu per minuta) — přetáčení je čisté krájení typed arrays. Timestampy se normalizují (`canonicalTs` — Arrow epoch vs. JSON ISO).
- **Stav:** React kontexty `AppState` (status z WS + REST initial fetch, view, téma, alerty) a `Crosshair` (sdílený všemi panely).
- **OI fallback:** při nulovém OI staví heatmapu z volume (engine mezitím posílá alert `oi_missing`).
- Wiki/manuál: statické HTML v `frontend/public/manual/` (generované z MD, viz níže) — servíruje ho vite dev i nginx.

9. Vývojové prostředí

Prerekvizity: [uv](#) (stáhne Python 3.12 sám), Node.js ≥ 20 , Docker (pro PG integrační test lokálně volitelně).

```
uv sync --all-packages                # Python workspace (engine + api)
uv run ruff check .; uv run ruff format .  # lint/format
uv run mypy engine/src engine/tests api/src api/tests
uv run pytest                          # PG integrační test se přeskočí bez GEXLENS_TEST_PG_DSN

cd frontend; npm ci; npm run lint; npm test; npm run build
```

Dev servery: `make run-api` (uvicorn :8000), `make run-frontend` (vite :5173), `make run-engine` (vyžaduje TWS). CORS povoluje i :5173.

Regenerace manuálů (MD → HTML pro in-app wiki → PDF): `powershell scripts/build-manual.ps1` (vyžaduje Edge; PDF vzniká headless tiskem).

Konvence: feature branch `feat/{issue}-slug` / `fix/...`, PR s `closes #N`, merge po zeleném CI. Výpočty vždy s golden testy v `engine/tests/golden/` (ručně spočtené hodnoty, výpočet dokumentovaný v `description`).

Oddělená prostředí DEV a PROD (#568)

Vedle produkčního stacku (`compose.yml`, :8080) existuje dev stack (`compose.dev.yml`, projekt `gexdev`, :8081) s vlastním PG volume (`gexdev_pgdata`) a vlastní kopií parquet dat (`data-dev/`). Cíl: vývoj se nikdy nedotkne produkčních dat, která nejdou znovu pořídít (věčný OI archiv, setupy, track record).

Skript / ikona	Co dělá	Souběh s prod
<code>scripts/start-prod.ps1</code> (ikona GEXLens)	produkce; shodí dev-live, pokud běží	—
<code>scripts/start-dev.ps1</code> (ikona GEXLens DEV)	dev bez enginu: PG + API + frontend nad kopií dat	povolen — market data účtu se nedotkne, prod dál sbírá
<code>scripts/start-dev.ps1 - Live</code> (ikona GEXLens DEV+Engine)	plný stack proti TWS	zakázán — skript nejdřív shodí produkci (jeden účet); po dobu běhu prod nesbírá

Skript / ikona	Co dělá	Souběh s prod
<code>scripts/start-dev.ps1 -LiveTasty (start-gexlens-dev-live-tasty.cmd)</code>	dev engine jen s tastytrade (#623): IBKR vypnuto, žádné výpočty ani zápisy — stream chainu do cache s minutovým heartbeatem v logu engine	povolen — tasty snese souběžné streamy (ADR-0027), produkce nepřijde o minutu; news-engine se nespustí (mluví s TWS)
<code>scripts/seed-dev.ps1</code>	obnoví dev PG z nejnovější zálohy + zrcadlí <code>data/</code> → <code>data-dev/</code>	povolen

Pravidla:

- **Produkce použít výhradně main** . `start-prod.ps1 -Build` odmítne stavět z jiné větve nebo ze špinavého stromu (`-Force` = vědomé obejít). Bez `-Build` se jen startují dřív postavené image. Dev použít libovolnou rozpracovanou větev.
- **Image se staví v CI, ne doma (#1139, 13. 9. 2026)**: po mergi do `main` workflow *Images* postaví `ghcr.io/kechicz/gex-python` (engine, api, news-engine) a `ghcr.io/kechicz/gex-frontend` (tagy `latest` + `sha-<7>`, label `org.opencontainers.image.revision`). Doma se image jen stahují (`docker compose pull`), lokální build zůstává nouzový (`deploy-engine-offhours.ps1 -Build`, `start-prod.ps1 -Build`). Balíčky jsou **veřejné** (repo je veřejné, veřejné balíčky jsou zdarma bez limitu; soukromé by narazily na 500 MB) — stažení nevyžaduje přihlášení. Jednorázově po prvním pushu: na stránce balíčku (GitHub → Packages → gex-python / gex-frontend → Package settings → Change visibility → Public). Do image jde jen to, co Dockerfile kopíruje; `.env*`, `data/`, `docs/` vylučuje `.dockerignore`.
- **Nasazení po mergi**: `git checkout main && git pull`, pak `.\scripts\deploy-engine-offhours.ps1` (stáhne image, počká na revizi HEAD, restartuje v okně). Starší cesta `.\scripts\start-prod.ps1 -Build` staví lokálně. Před nasazením, které sahá na schéma DB, vždy `.\scripts\backup-postgres.ps1` — izolace dev to nenahrazuje, je to druhá vrstva. Výchozí cíl záloh je od #439 (26. 8.) `D:\Programy\GEX\zalozy-pg` (dumy zabíraly 3,6 GB na systémovém C:), vlastní složka přes `-Target`.
- Dev frontend nese v sidebaru oranžový badge **DEV** (build arg `VITE_GEXLENS_ENV`), ať se okna prohlížeče nespletou.
- Dev stack je jednorázový: rozbitý dev = `docker compose -f compose.dev.yml down -v`, smazat `data-dev/`, `seed-dev.ps1` ZNOVU.
- Dev engine má výchozí `clientId 2` (`GEXLENS_DEV_IBKR_CLIENT_ID`), aby se v TWS nepotkal s produkční jedničkou.
- `-LiveTasty` si bere konzervativní strop 2 000 subskripcí (`GEXLENS_TASTY_MAX_SUBSCRIPTIONS` ; měřeno 6 008 bez degradace, ADR-0027) — kapacita je pravděpodobně per účet, dev nesmí ujídat produkci. Vyžaduje dev tasty grant v `.env.dev` (#696). Pozor: cross-feed

logika (shadow #613, fallback #614) se v tomto režimu ověřit nedá — potřebuje oba feedy vedle sebe.

10. Testy a CI

- **Python** (~160): jednotkové + golden (GEX, levels, heatmap módy, walls, CumΔ, profil), mock-based integrační (scheduler, hot zóna, runtime), PG integrační (v CI přes service kontejner), **e2e smoke** — deterministický referenční den přes celou pipeline engine→storage→API proti golden hodnotám.
- **Frontend** (~58): jednotkové (geometrie, barvy, contours, slice), komponentové (jsdom + testing-library, PointerEvent polyfill), Arrow round-trip loaderu, **e2e render smoke** (App nad /replay balíkem), vizuální regresní snapshoty renderu.
- **CI** (GitHub Actions, na každý PR): python job (ruff, mypy strict, pytest + PostgreSQL service), frontend job (eslint, prettier, vitest, build). Výkonnostní testy s tvrdým limitem běží jen lokálně (`ci` env skip).

Bezpečnostní kontroly v CI

Job `security` na každém PR: gitleaks (celá historie — pozor, test s realisticky vypadajícím tajemstvím spadne i po přepsání souboru, dokud je v historii větve), pip-audit, npm audit. Lokálně `pwsh scripts/security-scan.ps1`.

11. Známé limity účtu a otevřené body

Z [ADR-0001](#) (měřeno živě na účtu):

Limit	Hodnota	Dopad
Tick-by-tick streamy	5	Bez použití od ADR-0032 (3. 9. 2026): klasifikaci agresora dodá dxFeed <code>TimeAndSale</code> bez limitu; IBKR tick-by-tick zóna zrušena.
Market data lines	100	Naměřený strop účtu (sonda #609). Původní údaj „≥ 150“ v ADR-0001 neplatí — dávka 80 jede blízko stropu (~95–100/100), <code>batch_size</code> proto nezvyšovat . Strukturální řešení přinese #616.

Limit	Hodnota	Dopad
FOP OI	tick 588 nedodává nikdy; tick 101 funguje	VYŘEŠENO (issue #65, ADR-0001 v3): <code>IbOIFetcher</code> používá generic tick 101 pro OPT i FOP a čte hodnotu podle strany kontraktu (opačná strana = validní 0.0). Retry à 30 min + volume fallback zůstávají jako pojistka.

[ADR-0002](#): obálka strikes je grow-only (křídla se neztrácejí), strop šířky s alertem. [ADR-0003](#): multi-instrument orchestrace řízená watchlistem.

Sekundární datový zdroj — tastytrade/dxFeed (M7)

Naměřené limity a pasti feedu: **ADR-0027** (6 000+ symbolů na subskripci, REST $\geq$ 6 req/s, povinné KEEPALIVE, dekádoová kolize futures candle symbolů). Přístup výhradně **OAuth2 scope** `read` — nikdy `/sessions`, nikdy `trade` (ADR-0025); granty oddělené pro dev a produkci. Shadow mód (#613) porovnává oba feedy do `feed_comparison`, nic nepublikuje; vyhodnocení `scripts/feed_comparison_report.py`.

Konfigurace je jen přes `.env`, v Settings tastytrade není. Od fáze 2 (#614) se ale aktivní fallback **ukazuje v hlavičce** jantarovým chipem — tiché přepnutí zdroje zakazuje ADR-0025 pravidlo 5.

Pozor na klíče, které engine nečte. Model `Settings` má `extra="ignore"`, takže neznámý klíč se **tiše zahodí**. `GEXLENS_TASTY_CLIENT_ID` z `.env.example` nepoužívá nikdo (refresh flow posílá jen `refresh_token`

- `client_secret`) a `GEXLENS_DEV_TASTY_*` čte napřímo z prostředí jen sonda `scripts/tasty_probe.py`. Když nastavení „nezabírá“, ověř nejdřív, že ten klíč engine vůbec zná — seznam je v kapitole 4.

Vydání a odvolání grantu (#620)

Granty jsou **dva na jednom účtu** — zvlášť pro dev a zvlášť pro produkci (ADR-0025). Jde je odvolat nezávisle, takže zabití dev přístupu nechá produkci běžet dál.

Vydání

1. V tastytrade **Manage** → **Create Grant**, potvrdit druhým faktorem.
2. Zaškrtnout **výhradně scope** `read`. `trade` se nezaškrtně **nikdy** — nejde o důvěru v kód, ale o to, že právo, které aplikace nemá, nelze zneužít. `openid` jen tehdy, pokud ho autorizační tok vyžaduje.

3. **Client secret se ukáže jen jednou.** Kdo ho v tu chvíli neuloží, musí vydat grant znovu — zpětně se přechíst nedá.
4. Hodnoty zapsat do `.env` (produkce), resp. `.env.dev` (dev), pod klíči `GEXLENS_TASTY_CLIENT_SECRET` a `GEXLENS_TASTY_REFRESH_TOKEN`. Nikdy do repa ani natvrdo do compose. Před zápisem ověř, že soubor **končí novým řádkem** — append bez něj přilepí klíč k předchozí hodnotě.
5. Restartovat dotčený stack. Refresh token neexpiruje, access token (15 min) si engine obnovuje sám.

Ověření, že grant platí

co	kde	očekávané
spojení	<code>GET /status</code>	<code>tasty_connected: true</code>
subskripce	log enginu	<code>DXLink subskripce kompletní: N symbolů</code>
politika přístupu	<code>pwsh scripts/security-scan.ps1</code>	sekce <i>tastytrade přístup</i> bez nálezů

Odvolání

V **Manage** odvolat příslušný grant. Engine tím přijde o obnovu access tokenu, tastytrade větev přestane fungovat a s ní i křížová kontrola (#517 A), oba fallbacky (#614) a OI fill (#664) — IBKR cesta běží dál. Aktivní zdroj je vidět v hlavičce jantarovým chipem, takže ztráta nezůstane tichá (ADR-0025 pravidlo 5).

Rotace při podezření na únik: nejdřív **odvolat**, teprve pak vydávat nový. Opačné pořadí nechává uniklý token platný po celou dobu, kdy se vyplňuje `.env`.

Trvalé × dočasné je od #763 rozdělené. Do té doby hlídal jeden flag `GEXLENS_TASTY_SHADOW` obojí, takže nejpřirozenější možná úvaha — „měření doběhlo, vypínám ho“ — tiše vypnula i křížovou kontrolu, oba fallbacky, OI fill a publikaci ceny bez pipeline. Nově:

- `GEXLENS_TASTY_ENABLED` drží **trvalou** větev (default `true`),
- `GEXLENS_TASTY_COMPARISON_WRITE` jen **dočasný** zápis do `feed_comparison`.

Až doběhne vyhodnocení M7 fáze 2, vypíná se **druhý** z nich. Monitor běží dál a dodává tally detektoru i fallbackům; ověřuje to test, který porovnává tally se zapisováním a bez něj — musí vyjít shodně, jinak by se aplikace po konci měření začala rozhodovat jinak.

Provozní stav tasty větve (reconnecty DXLink, handshake, počet sledovaných symbolů, zapsané řádky) končí **jen v logu kontejneru** a v tabulce `feed_comparison`; V `/status` je z něj `chain_source` a `spot_source`. Samostatná obrazovka je zadaná v #706.

Křížová kontrola feedů (#517 fáze A)

Heartbeat testuje jen TCP spojení a stall detektory (ADR-0015) měří pasivně stáří dat — incident 26.–27. 7. (15 h zmrzlé ATM greeks při tekoucích cenách) nezachytila ani jedna vrstva. Chyběla **nezávislá reference**: mlčí data, nebo mlčí trh? Běžící tasty větev ji dodává zadarmo, takže detektor nedělá **žádný request na IBKR a nebere žádnou market data line** — jen čte, co `compare_minute` stejně počítá.

Rozhoduje se z rozpadu kontraktů podle čerstvosti obou stran:

Situace	Verdikt	Chování
IBKR mlčí $\wedge$ tasty data má	<code>ibkr_suspect</code>	alert — tasty vylučuje „tichý trh“; farmu od mrtvých subskripcí rozliší až sonda fáze B
oba mlčí	<code>quiet</code>	ticho — nikdo neobchoduje, není co hlásit
tasty mlčí $\wedge$ IBKR data má	<code>tasty_suspect</code>	jen stav a log, bez alertu — viz níže
oba dodávají	<code>ok</code>	—

Proč zrovna 70 % / 3 minuty. Sweep obchází kontrakty po dávkách `batch_size`, takže **každou třetí minutu** vyskočí podíl „IBKR mlčí“ na ~58 % a hned zase spadne — rotační artefakt, ne porucha. Okamžitý podíl je proto jako signál nepoužitelný a naivní práh by alertoval 24/7. Měření na 3 016 minutách čisté shadow historie (13.–16. 8. 2026) — nejdelší souvislá série minut nad prahem:

práh	nejdelší série	počet epizod
50 %	2 min	696
60 %	2 min	42
70 %	2 min	2
80 %	1 min	1

Série tří minut v řadě nenastala při žádném prahu ani jednou. Podmínka „M minut v řadě“ tedy rotaci odfiltruje úplně; `_MINUTES` snížené na 2 by naopak plašilo.

Proč `tasty_suspect` nealertuje. Přehrání téže historie detektorem dalo 41 epizod „tasty mlčí $\wedge$ IBKR data má“ — všechny v hodinách **21–06 UTC**. Příčina je konstrukční: dxFeed je **event-driven** (v klidu neposílá nic, okno stáří vyprší), IBKR sweep **poll-driven** (vrací poslední kotaci

pořád dokola). V noci a v denní pauze CME (16:00–17:00 CT) je to normální stav, ne porucha — v alert kanálu by z toho bylo 41 planých poplachů. Stav se proto jen poznamená do `/status` a jednou na začátku epizody do logu.

Kontrolní měření po téhle úpravě: **3 053 minut čisté historie → 0 alertů.**

Stav je v **Settings** → **Stav enginu** (řádek *Křížová kontrola feedů*) a v `/status` jako `feed_crosscheck`. Když tasy větve neběží, pole ve statusu **chybí** — UI to ukáže jako „neměří se“, což je jiný stav než `ok`.

Fallback na tastytrade (#614 fáze 2a a 2b)

Když IBKR přestane posílat data, přebírá je tastytrade. Typická příčina není porucha, ale **souběh s mobilem**: market data jsou per uživatel, takže přihlášení do IBKR aplikace v telefonu přetáhne feed k sobě (error 10197) a engine zůstane připojený — jen mu přestanou chodit ticky.

Co	Kdy nastoupí tasy	Kdy se vrátí IBKR
Spot podkladu (2a)	30 s bez ticku	60 s souvislých dat
Opční řetěz — kotace, greeks, OI (2b)	verdikt <code>ibkr_suspect</code> , tedy 3 min	5 čistých minut v řadě

Řetěz se spouští **verdiktem křížové kontroly**, ne vlastním prahem — dědí tak kalibraci měřenou na 3 016 minutách místo nového odhadu. Návrat vyžaduje `state == "ok" AND streak == 0`: samotné `ok` nestačí, protože detektor ho vrací i pro minutu nad prahem, která zatím nenaplnila sérii do alertu. Stav `quiet` a `insufficient` sérii jen nulují — tichý trh není důkaz uzdravení.

Co fallback vědomě NEoddá: kumulativní denní objem. Tasy ho ve stejné sémantice nemá, a dosadit nulu by znamenalo skokový záporný přírůstek přes celý řetěz. Po dobu fallbacku řetězu proto **stojí CumΔ a net objem** a ve snímcích jsou `NULL` — díra, kterou je vidět, místo čísla, které lže. Heatmapa, GEX profil, zdi i flip běží dál.

Přepnutí se hlásí alertem a jantarovým chipem v hlavičce; `/status` nese `chain_source` a `spot_source (ibkr | tasty)`. `spot_source` se agreguje pesimisticky — stačí jeden instrument na fallbacku a status hlásí `tasty`, protože výpadek market data je vlastnost účtu.

Slepé místo „mrtvá tasy větve selže tiše“ uzavřelo #764: `tasty_suspect` sám o sobě dál nealertuje (41 planých epizod v noci), ale rozlišovač `GEXLENS_CROSSCHECK_CHANGE_THRESHOLD` (kap. 4) pozná, že se trh hýbe (IBKR hodnoty se mění) a tasy přitom mlčí → alert `feed_backup_dead`. Záloha, která umřela za běhu, se tak pozná dřív než při výpadku IBKR.

Degradovaný start a co ještě přebírá tasty (v1.17, #1153, ADR-0025 dodatek 14. 9.)

Fallback výše řešil jen **běžící** pipeline. Od 14. 9. přechod na tasty platí i tehdy, když IBKR vypadne dřív, než se pipeline založí (Error 1100 = TWS bez spojení s IBKR, nebo Gateway vyhozená souběhem s mobilem — API port odmítá):

- **Cache discovery** `data/derived/discovery_cache.json` — per symbol front futures kontrakt + všechny expirace a striky z posledního úspěšného IBKR discovery (přepisuje se při každém úspěchu). Při selhání discovery nebo bez API socketu se z ní pipeline založí („degradovaný start“, alert `degraded_start`, log `Degradovaný start ES (#1153): ...`), úvodní spot dá tasty, řetěz/OI/svíčky fallbacky. Záznam se nepoužije, když je starší než 14 dní, front kontrakt expiroval nebo nemá dnešní expiraci → pipeline pak čeká na IBKR (čistá instalace, roll během výpadku).
- **Cyklus pipeline bere spot z tasty** (`spot_override`), takže GEX ani hlídač barů nepočítají nad cenou z doby výpadku.
- **Svíčky podkladu**: při stall (3 min bez real-time barů při živém spotu) engine každou minutu doplní chybějící minuty z dxFeed Candle (`source = tasty_candle` , banner „doplněno“ v UI); jedno pomocné DXLink spojení naráz (souběžný fetch ES+NQ nechával druhý bez svíček).
- **Tasty subskripce řetězu** se plánují z kontraktů, které pipeline drží, a pro symboly z konfigurace i watchlistu bez pipeline; kontrakt bez IBKR kotace je pro křížovou kontrolu mrtvá IBKR strana (jinak by po restartu bez IBKR hlásila „sledováno 0 kontraktů“ a fallback řetězu se nezapnul).
- **Stáří tasty hodnot = živost streamu** (`GEXLENS_TASTY_CHAIN_MAX_AGE_S` je práh posledního eventu STREAMU): dxFeed posílá jen změny, deep OTM strike nezměněný 20 min je na živém streamu aktuální. Kontrakt bez dxFeed Greeks dostane BS greeks z mid (`greekssource = computed`), bid 0 zůstává ve snímku (limitně nulové greeks) — jinak ze strike profilu mizelo OI těch striků.
- **IBKR volání mají stropy**: discovery podkladu 3×30 s, discovery řetězu 30 s, kvalifikace kontraktu 20 s a po timeoutu 60 s fail-fast (do 14. 9. `qualifyContractsAsync` bez stropu držel OI archiv i celý orchestrátor).

Diagnostika: `/status.spot_source` , `chain_source` , `feed_crosscheck` , `tasty_symbol_breakdown.chain` (má odpovídat počtu držených kontraktů, 0 = fallback nemá z čeho brát); log `Cyklus ES ...: N snapshotů` má za fallbacku odpovídat obálce (ES 160, NQ 96; `greeks 0/160` v tomtéž řádku počítá jen IBKR modelGreeks, tasty greeks jsou ve snímku).

Start bez běžícího TWS (#756)

Engine na IBKR čeká **nejvýš** `GEXLENS_STARTUP_CONNECT_WAIT_S` (default 60 s) a pak rozjede zbytek i bez něj. Do #756 tu bylo nekonečné čekání, za kterým zůstalo úplně všechno — schéma DB, pipelines i tastytrade větev — takže po startu Windows bez spuštění TWS neběželo nic.

Co platí po vypršení stropu:

- pipeline se zakládá jen **degradovaně** z cache discovery + tasty spotu (v1.17, #1153 — viz výše); bez cache se nezakládá; existující se **neruší** — `plan.stop` se řídí watchlistem, ne spojením,
- tasty se odebírá i pro instrumenty **bez** pipeline, jinak by fallback neměl z čeho stavět,
- cenu publikuje samostatná smyčka, která pro symbol s běžící pipeline umlkne,
- `/status` chodí i bez pipelines a `connection` nese **skutečný** stav — „engine běží“ a „IBKR je připojen“ jsou dva různé stavy.

Pipeline se založí sama, jakmile se spojení objeví. Restart enginu není potřeba.

12. Diagnostika a údržba

Situace	Postup
Engine offline	<code>docker compose logs engine</code> — hledej stav ConnectionManageru; ověř TWS (API zapnuté, port, Trusted IP). Warning 2110/2103 = výpadek TWS↔IB, vyřeší se sám.
Prázdné GEX/walls	Zkontroluj <code>oi_eod</code> pro dnešek: <code>docker compose exec postgres psql -U gexlens -c "select date, count(*) from oi_eod group by 1 order by 1 desc limit 5"</code> — pokud dnešek chybí, engine archiv opakuje à 30 min (CME publikuje OI ráno).
Ticker z watchlistu nesbírá	<code>`docker compose logs engine</code>
Vysoké <code>Repair</code> / <code>Stale</code>	Konkrétní kontrakty bez dat — často nelikvidní křídla; zvyš <code>BATCH_TIMEOUT_S</code> nebo zmenš obálku.
Disk roste	Retention běží nočně; ručně: smaž staré partice v <code>./data</code> (nikdy <code>oi_eod</code>).
Panel Sentiment plochý na nule (ES/NQ)	Ověř váhy: <code>docker compose exec postgres psql -U gexlens -c "select symbol, category, predictor, round(weight::numeric,3) from news_weights order by 1,2,3"</code> — od ADR-0036 (#1150) je váha vždy v [0,25; 2,0],

Situace	Postup
	mince = 1,0; samé nuly znamenají image před #1150. Řadu zpětně spraví <code>docker compose exec news-engine python -m gexlens_news recompute-sentindex --from YYYY-MM-DD [--to YYYY-MM-DD]</code> (přepíše partice <code>derived/sentiment/{SYM}/</code> i svíčky <code>sentiment_daily</code> toutéž mechanikou jako živý job, σ/close_z od <code>--from</code> dál dopočte znovu; dnešek jen do „ted“). Bez eventů se skórem (<code>select count(*) from news_events where sentiment_score <> 0 and ts_event > now() - interval '1 day'</code>) je problém v klasifikaci, ne ve vahách.
Reset prostředí	<code>docker compose down</code> , smaž <code>./data</code> (přijdeš o 14denní okno, ne o OI archiv ve volume <code>pgdata</code>), <code>docker compose up -d --build</code> .
Málo dat po restartu	Writer navazuje na rozepsaný den — mezera zůstane jen za dobu výpadku.
Souběh s mobilem: graf stojí déle než ~4 min	<code>/status</code> → <code>spot_source / chain_source</code> mají být <code>tasty</code> , <code>tasty_symbol_breakdown.chain > 0</code>; log <code>grep -E "Degradovan Fallback řetězu Rekonstrukce Setup .* selhal"</code> . <code>Setup ... nedorazila cena podkladu</code> = <code>tasty</code> nemá čerstvý spot symbolu (front future není v subskripci — od #1163 se odebírá i pro watchlist); 0 snapshotů s <code>chain_source: tasty</code> = chybí <code>tasty chain</code> subskripce (<code>chain: 0</code>). Cache discovery: <code>python -c "import json; print(json.load(open('data/derived/discovery_cache.json')).keys())"</code> . Nerestartovat engine bez příčiny — restart během výpadku IBKR = degradovaný start (~2–3 min).
Potřebuju log enginu z doby PŘED restartem/deployem	Log kontejneru zmizí s jeho recreate. <code>scripts/deploy-engine-offhours.ps1</code> ho od #1056 (v1.6) ukládá sám do <code>data/logs/engine-<YYYYMMDD-HHMM>.log</code> (a <code>...-crashed.log</code> před rollbackem; bez uloženého logu se nenasazuje; retence 30 dní). Při ručním <code>docker compose up -d engine / --force-recreate</code> udělej totéž předem: <code>docker logs gex-engine-1 > data/logs/engine-\$(Get-Date -Format yyyyMMdd-HHmm).log 2>&1</code> . 7. 9. (#1054) se bez toho hodinu řešil falešný poplach.
Docker roste / disk D: dochází / Docker Desktop se nerozjede	<code>pwsh scripts/docker-cleanup.ps1</code> (rollback tagy, build cache > 7 d, dangling; <code>-whatIf</code> jen ukáže) → když VHDX zůstane velký, <code>pwsh -File scripts/compact-docker-vhdx.ps1</code> jako správce při zavřeném trhu (12. 9. 2026: 70,9 → 24,9 GB). Nikdy <code>docker volume prune</code> ani reset Docker Desktopu — Postgres je uvnitř VHDX. Podrobně kap. 13.4.

Situace	Postup
Díra ve snapshotech, ale bary v okně jsou	Podívej se na <code>source</code> barů v <code>derived/{sym}/bars/ : ibkr_hist</code> = doplněno z IBKR historical při dalším startu → engine v tu dobu neběžel (vypnuté PC, zastavený kontejner), ne stall řetězu. Stall vypadá obráceně: bary <code>ibkr</code> tečou, snapshoty chybí. Do #1055 (8. 9. 2026) se backfill tvářil jako <code>ibkr</code> a 2,5 dne vypnutého PC vyvolalo falešný poplach (#1054).
Změna portu TWS	Settings v aplikaci (platí do sekund i bez spojení, #992), a zároveň <code>.env</code> + <code>docker compose up -d engine</code> — hodnota v DB přebíjí <code>.env</code> , takže samotná změna <code>.env</code> skončí skokem zpět na starý port (engine to hlásí <code>WARNING: Nastavení připojení ze Settings UI (DB) přebíjí .env</code>).
Zaseknuté spojení, restart kontejneru nechci	Settings → Stav enginu → Přepojit IBKR / Přepojit tastytrade (#950) — 1–2 min díra, mimo US RTH. Uložení nastavení beze změny hodnot nepřepojuje.
Po restartu Docker Desktop frontend vrací 502 (Settings nejde uložit, watchlist prázdný)	Do #993 nginx držel starou IP služby api; nově se resolvuje za běhu (do ~10 s). U staršího image <code>docker compose restart frontend</code> , trvale rebuild frontendu.
Alert <code>greeks_stalled</code> každý den po settle	Do #959 hlídka běžela i nad expirující řadou, která se po settle přestane kotovat (93 % chybějících Greeks, sekundární řada plná). Nově se po settle dané expirace nehlásí.
<code>subscription size is too big</code> v logu / <code>tasty_budget.size_exceeded</code> roste	Strop 25 000 položek <code>symbol x event</code> na spojení (#982, kap. 4). Zkontroluj <code>/status.tasty_budget</code> — které složky se ořezaly; ad-hoc pohled má rezervu <code>GEXLENS_TASTY_ADHOC_RESERVE_ENTRIES</code> . Heal se na tuhle chybu záměrně nespouští.
Reddit 429 v logu news-engine	Limit per IP napříč subreddity (#941) — kolektor jede round robin, jeden subreddit za cyklus à 300 s, retry při 429. Ojedinělé 429 jsou normální; trvalé = zkrátil se <code>GEXLENS_NEWS_REDDIT_RSS_FEED_DELAY_S</code> nebo interval.
Signály vznikají mimo obchodní dobu / bez outcomes	Do #968 vznikaly i se zavřeným trhem (21 z 39, žádný neměl outcome — bez barů není z čeho měřit). Generování je vázané na <code>compute/marketclock.is_market_closed</code> ; expirace a dopočet outcomes běží dál. Latentní past: <code>_evaluate_outcomes</code> čte jen 3 dny zpět.

Situace	Postup
Databáze roste, pg_database_size v GB	Největší tabulky vypisuje alert disk_space (#773). feed_comparison je od #965 zhuštěná (kap. 3); trades/ a snapshots/ v parquetu jsou keep-forever záměrně (ADR-0029).
Graf zamrzl, engine hlásí connected	Souběh s mobilem (error 10197) — market data jsou per uživatel. Od #614 přebírá data tastytrade: do 30 s cena, do 3 min celý řetěz; v hlavičce svítí jantarový chip. Zamrzne-li i tak, ověř tasty větev: `docker compose logs engine
CumΔ stojí, zbytek jede	Očekávané chování při fallbacku řetězu (#614) — tasty denní objem v sémantice IBKR nedodává, tak se nevymýšlí. Skončí návratem na IBKR.
V logu není ani řádek tasty	Chybí grant v .env (GEXLENS_TASTY_CLIENT_SECRET , _REFRESH_TOKEN) nebo je vypnutý GEXLENS_TASTY_ENABLED (od #763; zastaralý GEXLENS_TASTY_SHADOW , je-li nastaven, ho přebíjí a engine to hlásí varováním), viz kap. 11.
Error 200, reqId 5/6 po startu	Známé (#734): BRFPD a DJ nemají celofeedovou pásku. Není to porucha, zprávy tečou z BRFG a DJNL .
Zvonek greeks_bs_fallback / vysoké greeks_bs_share ve /status	TWS přestala dodávat model greeks a engine je dopočítává BS modelem z mid (#547) — data tečou dál, jen z jiného modelu (přesně bouře #862: 29 h na BS, vyléčil až restart TWS). /status.greeks_bs_share nese podíl per symbol; alert přijde po 15 min epizody nad 20 %. Při plném fallbacku (>80 % ≥ 30 min) zasáhne engine sám (#877, varianta C): 1. pokus vynucená obnova subskripce (bez díry), 2. pokus reconnect — výhradně mimo US RTH ; každý pokus hlásí zvonek. Vypnutí zásahů: GEXLENS_BS_FALLBACK_RECONNECT=false (alert zůstává). Nepomůže-li ani reconnect, restartuj TWS.
DXLink ERROR ... subscription rate is too high v logu	Server odmítl dávku subskripce a odmítnuté symboly by tiše mlčely. Od #863 se engine léčí sám: rozestup dávek se zdvojnásobí (0,25 s → strop 2 s) a po 10 s klidu pošle znovu — od #936 jen mlčící symboly (cache.silent_symbols , event starší 10 min nebo žádný; typicky jednotky dávek), plný resubscribe jen když mlčí většina (výpadek); plný heal za RTH trhal limit znovu a nekonvergoval (79 healů / 2 h, 5 881 rate limitů / den). Rozestup se po 120 s klidu vrací k základu. /status.tasty_rate_limited počítá odmítnutí, tasty_heals

Situace	Postup
	healy; roste-li trvale i s pomalým tempem, je problém jinde (kapacita účtu — nebo strop položek, viz <code>size_exceeded</code>). Transportní <code>ping_timeout</code> je 120 s (#916): default 20 s byl kratší než plná subskripce na stropu rozestupu (~44 dávek à 2 s ≈ 90 s) a klient spojení zabil dřív, než doběhla → smyčka reconnectů a fallback #614 bez dat; mrtvé spojení hlídá protokolový KEEPALIVE (60 s) + backoff.
<code>tasty_symbols</code> výrazně nad plán ADR-0025	<code>/status.tasty_symbol_breakdown</code> nese rozpad per účel (<code>chain</code> / <code>extended</code> / <code>wide</code> / <code>underlying</code> / <code>total</code>) — podle něj se pozná, která složka přerostla (typicky souběh řetězů více expirací během dne; v noci klesá).
Chyby 354/300 s <code>neznámý kontrakt</code> v <code>subscription_error_recent</code>	Do 27. 8. anonymní: náhrobky reqld vzorkovaly jen <code>mktData</code> , kdežto tick-by-tick hot zóny se registruje pod <code>AllLast</code> / <code>BidAsk</code> . Nově nesou popisek i <code>[tickType]</code> — podle něj poznáš rotaci hot zóny (ATM se pohnul) od výpadku kotací.
Stínové CumΔ z TimeAndSale (#615 fáze 3)	Paralelní měřicí řada z dxFeed <code>aggressorSide</code> — živé CumΔ ani detektory se NEmění. Minutové řádky v <code>derived/{sym}/cumdelta_dx/</code> (od ADR-0032 doplnku 4. 9. celý sbíraný řetěz bez zón; <code>flow_hot</code> zůstává 0), <code>/status.tasty_dx_shadow</code> nese denní <code>unknown_side_share</code> , <code>volume</code> a od #1070 (v1.6) <code>seeded_from_ts</code> — po restartu uprostřed seance se kumulativ navazuje z posledního řádku partice (do 8. 9. 2026 začínal od nuly a den měl tolik řetězů, kolikrát engine startoval; řádky téže minuty se upsertují). Živá trade větve (#615 fáze 3, v1.4): <code>CumDeltaTracker.add_dx_trade</code> přijímá tytéž tisky; při <code>GEXLENS_CUMDELTA_SOURCE=midpoint</code> jen měří pokrytí (<code>/status.cumdelta_coverage</code> : <code>printed_share</code> , <code>structured_volume</code> , <code>fallback_volume</code>), při <code>dxfeed</code> dává znaménko toku a bar větve klasifikuje midpointem jen tisky bez strany a kontrakt-minuty bez tisků; řada <code>flow/</code> nese sloupec <code>source</code> . Spread legy se nerozlišují (3. 9. 2026, ADR-0027 doplněk): CME příznak <code>spreadLeg</code> pro FOP nenese, řada „outright“ a <code>spread_volume_share</code> zrušeny. Vypínání: <code>GEXLENS_TASTY_DX_CUMDELTA=false</code> .
Kandidát T9 „strop nad hlavou“ (#577, fáze 1)	Tichý sběr výskytů do PG <code>setup_probes</code> : vstup do tlumící zóny zespodu (t9_ceiling, hypotetický LONG hrana→střed) a výpad dolů (t9_exit, SHORT o šířku zóny); kotvy z geometrie pásma #575 (žádné body/ATR — poučení #434), akceptace 5 min, vyhodnocení týmž

Situační	Postup
	evaluate_bar jako živé setupy, timeout na settle expirace runtime (do 4. 9. settle kalendářního dne — výskyt po 20:00 UTC se zavřel další minutou). Rozhoduje čistá funkce <code>compute/setups.detect_damping_ceiling</code> (mimo <code>detect_all</code>, regresní zámeček <code>test_setups_regression.py</code>); tutéž spouští <code>python scripts/backtest_setups.py --probes --data <kořen dat></code> nad historií (bary + <code>gexprofile/</code>, bez DB) — report per instrument, šablona a směr. Do <code>setups</code>, alertů ani track recordu nejde nic; fáze 2 při ≥ 30 výskytech na instrument.
Ad-hoc pohled (#521 C)	Tabulka <code>adhoc_view</code> (PG) je most UI→engine; frontend prodlužuje <code>requested_ts</code> à 1 min, engine bez prodloužení pohled po ~3 min uklízí a subskripce vypadnou diffem. Data jdou výhradně z tastytrade (žádné IBKR linky); <code>/status.tasty_adhoc</code> nese aktivní produkty, rozpad subskripcí má složku <code>adhoc</code>. Od v1.4 pohled ustoupí produktu s plnou pipeline (<code>refresh</code> ho uklidí i s řádkem <code>adhoc_view</code>) a UI nepinguje, dokud nezná watchlist — 4. 9. vznikl ad-hoc NQ po restartu enginu (pipelines ještě prázdné) a přežíval díky pingům při každém otevření stránky. Partice ad-hoc symbolu uklízí běžná 14denní retence.
Zdroje zpráv: registr + audit (#578)	Tabulka <code>news_sources</code> (tier core/extra/test, očekávaný denní objem, enabled, notes; seed nepřepisuje ruční editace). Audit: <code>GET /news/sources</code> — realita per zdroj včetně neregistrovaných. Nové zdroje 27. 8.: bluesky (Jetstream firehose bez klíčů, klientský filtr cashtags+makro+kurátoři přes <code>GEXLENS_NEWS_BLUESKY_CURATED_AUTHORS</code>, strop 30/min, vypínání <code>GEXLENS_NEWS_BLUESKY_ENABLED=false</code>) a reddit_rss (nativní RSS wsbs+stocks hot à 300 s, <code>GEXLENS_NEWS_REDDIT_RSS_ENABLED=false</code> vypíná).
Walk-forward parametrů setupů (#794 fáze 3, ADR-0034)	<code>pwsh scripts/walkforward-nightly.ps1</code> — registrováno v Task Scheduleru jako úloha „GEXLens walk-forward“ (po-pá 23:30, zmeškaný start se dožene, <code>scripts/register-walkforward-task.ps1</code>; log <code>data/reports/walkforward-nightly.log</code>; bez běžícího <code>gex-postgres-1</code> se tiše přeskočí) → <code>data/reports/walkforward-<datum>.md</code>: replay archivu produkčním detektorem pro kandidáty z <code>configs/walkforward_grid.json</code> (baseline = platná verze <code>setup_params</code>), IS 20 / OOS 5 seancí, zábrany (≥ 20 OOS seancí, $\geq 50\%$ foldů, $\emptyset \Delta \geq 0,1$ R/den, Bonferroni p). Nic nezapisuje — případný návrh je tělo

Situace	Postup
	pro <code>POST /setups/params</code> , odesílá člověk. Přímo z hostitele: <code>uv run python scripts/walkforward_setups.py --db "postgresql+psycopg://gexlens:<heslo>@127.0.0.1:55432/gexlens" [--json] (heslo = GEXLENS_PG_PASSWORD ; URL z <code>.env</code> míří do compose sítě a z hostitele neplatí).</code>
Backfill zpráv	<code>python scripts/alpaca_news_backfill.py --dry-run</code> (změří objem), pak <code>-from 2024-07-28</code> — historie headlines z Alpaca s filtrem relevance (indexové ETF + mega caps; bez něj je ~85 % zpráv small caps bez vazby na index).

13. Zprovoznění od nuly — IBKR účet, TWS/Gateway

Jednorázový onboarding pro nové prostředí (převzato z issue #1, kde vznikl a byl odškrtnán při prvním zprovoznění 16. 7. 2026). Bez těchto kroků se engine nepřipojí, nebo dostane jen delayed data, která záměrně odmítá (SPEC 3.1 — Greeks z delayed dat nejsou spolehlivé).

13.1 Market data subskripce (Client Portal)

1. <https://www.interactivebrokers.com> → **Log In** → **Portal** (IBKR login + IB Key).
2. **Settings** → **User Settings** → **Market Data Subscriptions** → Configure (ozubené kolo).
3. **North America** → **Futures** → **CME Real-Time (NP,L2)** — pokrývá ES/NQ futures i futures opce (FOP). Levná subskripce (~1,55 USD/měs.) prokazatelně stačí (ověřeno živě, ADR-0001).
4. Zkontroluj status **Non-Professional** (jinak výrazně vyšší poplatky).
5. (Až pro SPY/SPX — sekundární scope): **OPRA (US Options Exchanges)**, pro SPX index navíc **Cboe Streaming Market Indexes**.
6. Vývoj proti **paper účtu**: Settings → Account Settings → Paper Trading Account → *Share real-time market data with paper account* — jinak paper účet subskripce nevidí.

13.2 TWS nebo IB Gateway (musí běžet lokálně)

Engine se připojuje socketem na lokální TWS/Gateway (z kontejneru přes `host.docker.internal`), ne přímo na servery IBKR.

Varianta A — stávající TWS (nejrychlejší): Edit → Global Configuration → API → Settings →  *Enable ActiveX and Socket Clients*, port **7496** live / **7497** paper, Trusted IPs `127.0.0.1` (vypne potvrzovací dialog), *Read-Only API* nechat **zapnuté** — GEXLens jen čte, nic neobchoduje.

Varianta B — dedikovaný IB Gateway (doporučeno pro trvalý provoz, #461):

1. Stable Windows 64-bit: <https://www.interactivebrokers.com/en/trading/ibgateway-stable.php>
2. Login obrazovka: režim **IB API** (ne IB Trader Workstation), Live/Paper, přihlášení s IB Key.
Pozor: přihlášení se stejným username **odhlásí běžící TWS** (a naopak, kap. 13.3) — přepojovat mimo seanci.
3. Configure → Settings → API → Settings:
 - port nechat **4001** live / **4002** paper (nepřepisovat na 7496 — po #737 poběží TWS vedle Gateway a port by kolidoval);
 - **Read-Only API** zapnuté;
 - **Allow connections from localhost only vypnout** — engine se připojuje z kontejneru přes `host.docker.internal`, ne z `127.0.0.1`; s výchozím nastavením se nepřipojí;
 - **Trusted IPs** zkopírovat z dnešní TWS (Edit → Global Configuration → API → Settings) — samotná `127.0.0.1` nestačí, spojení z Dockeru přichází z jiné adresy a bez trusted IP Gateway vyhodí potvrzovací dialog při každém reconnectu. U Docker Desktop (WSL2) je to `192.168.65.254` — přesnou adresu ukáže log enginu (`Connect call failed ('192.168.65.254', 4001)`) nebo potvrzovací dialog Gateway;
 - Master API client ID nechat prázdné.
4. Configure → Settings → Lock and Exit → **Auto restart**, čas mimo seanci (např. 23:00).
5. Přepojit engine — obojí, jinak se po příštím vytvoření kontejneru vrátí starý port:
 - v aplikaci Settings → IBKR → Port `4001`, uložit (engine se přepojí bez restartu a pipeline založí v nejbližším cyklu, #455);
 - V `.env` `GEXLENS_IBKR_PORT=4001`, pak `docker compose up -d engine` (předtím uložit log kontejneru, kap. 12 / 13.5 — #1056) (restart kontejneru nestačí, env se čte při jeho vytvoření). Host v `.env` neměnit — compose ho přepisuje na `host.docker.internal`. News-engine vlastní socket k TWS nemá, nic dalšího se nepřepojuje. Pořadí je jedno, ale **obojí je nutné**: hodnota ze Settings (DB) `.env` přebíjí — kdo změní jen `.env`, uvidí connect na 4001 a o sekundu později skok zpět na 7496 (#992; engine to od té doby hlásí `WARNING` em při startu).
6. Ověřit: `Test-NetConnection 127.0.0.1 -Port 4001`, stavová lišta `connected :4001` a `• Live`; v logu enginu zkontrolovat, že chodí broker news pásy (`reqMktData` broad tape).

Naměřená zátěž (2. 9. 2026, 55 min po startu, Globex před US RTH, ES+NQ, 3 × 60 s přes

`Get-Process ibgateway`):

	TWS (4. 8.)	IB Gateway	rozdíl
CPU klid (1 jádro = 100 %)	16 %	19 % medián, špičky ~38 %	srovnatelné
RAM	1 003–1 087 MB	651 MB	–40 %
CPU při konfliktu session (error 10197)	244 %	—	Gateway neřeší, jen druhý username (#737)

Gateway tedy šetří paměť, ne procesor; hlavní zdroj zahřívání PC (souběh loginů na jednom username) odstraní až druhý username. Provozní důvody (bez GUI, auto-restart, žádné okno na ploše) platí dál.

Denní a týdenní reautentizace (TWS i Gateway stejně): *Auto restart* restartuje aplikaci denně bez zadávání hesla. Týdenní cyklus IBKR začíná každé pondělí — po víkendovém resetu serverů se auto restart sám nepřihlásí a aplikace čeká na ruční login (potvrzení IB Key push); v logu `Login failed = Soft token=0 received instead of expected permanent`. Není to vlastnost Gateway, ale session na straně IBKR ([dokumentace IBKR](#)). U TWS to není vidět, protože se do ní během týdne přihlašuje kvůli obchodování tak jako tak. **V neděli po auto-restartu (nebo v pondělí ráno před seancí) potvrdit IB Key push**, jinak engine v pondělí nasbírá díru až do přihlášení.

Ověřeno v provozu (7. 9. 2026, #1016): týdenní cyklus platí přesně takhle — pondělní ruční přihlášení s IB Key bylo potřeba, zatímco denní auto restart ve 23:00 (4. → 5. 9.) i nedělní start seance (6. 9. 22:00) proběhly bez zásahu (bary ES 3.–7. 9. kompletně z IBKR, žádný `tasty_candle fallback`). Výpadek z noci 3. → 4. 9. (Gateway skončila ve stavu `PRELOGON`, engine 9 h na `tastytrade`) se neopakoval — byl jednorázový, ne vlastnost auto restartu. Provozní rutina je tedy **jednou týdně, v pondělí ráno před seancí**: otevřít okno Gateway, potvrdit IB Key push, zkontrolovat `connected :4001` (kap. 13.5).

13.3 Konflikt jednoho přihlášení ⚠

IBKR povoluje jedno přihlášení na username: Gateway + TWS (či mobil s trading permission, druhé PC) se stejným loginem současně = Gateway spadne. Řešení: druhý user v Client Portal (Settings → Account Settings → Users & Access Rights) — pozor, market data subskripce se platí per user. Pro start stačí varianta A.

13.4 Software na PC

- **Docker Desktop** s WSL2 backendem — celý stack (PostgreSQL, api, engine, frontend) běží přes `docker compose up -d` (kap. 3). Bez Dockeru: Python 3.12, Node.js ≥20, PostgreSQL 16 + `make run` (kap. 9).
- Volné místo: ~1 GB pro 14denní datové okno; WSL2 limit paměti viz `C:\Users\
<user>\.wslconfig` ( `[wsl2] memory=6GB` — pojistka proti nafouknutí vmmem). Od 9. 9. 2026 (#1105) navíc `autoMemoryReclaim=gradual` (VM vrací nevyužitou cache Windows, jinak vmmem sedí trvale na stropu) a `sparseVhd=true` (virtuální disk uvolňuje smazaná data); platí po `wsl --shutdown` nebo restartu PC.
- **Paměť procesů (#1105):** engine i news-engine logují RSS každých 10 min (`engine: RSS` 1234 MB) a engine ho hlásí do `/status.memory_rss_mb` (Settings → „Paměť enginu“). Při hledání viníka nastav `GEXLENS_MEMORY_TRACE=1` (tracemalloc, top-10 řádků kódu podle přírůstku; dražší běh, jen dočasně). Noc 10./11. 9. 2026 ukázala, že RSS roste mimo Python heap (glibc drží uvolněné bloky v arénách) — hlídka proto po každém vzorku volá `malloc_trim(0)` a loguje, kolik MB vrátila (`malloc_trim` vrátil N MB , vypnutí `GEXLENS_MALLOC_TRIM=0`), a compose nastavuje `MALLOC_ARENA_MAX=2` pro engine i news-engine. Vyhodnocení 15. 9.: u news-engine to stačilo (trim vrací 280 MB na vzorek, RSS z 3,1 GB na 0,4–0,7 GB), engine drží ~3 GB nezávisle na trimu (45 MB na vzorek) — proto hlídka od 15. 9. po každém vzorku loguje i **Arrow pool** pyarrow (`Arrow pool` (mimalloc) alokováno N MB, max M MB; `release_unused` vrátil K MB), volá `release_unused()` před glibc trimem a podíl hlásí do `/status.memory_arrow_mb` ; vypnutí `GEXLENS_ARROW_RELEASE=0` . Čtení: velký `release_unused` = cache Arrow (partice), malý = paměť drží Python objekty a numpy pole. **CPU a teplota** (11. 9. 2026, notebook i5-10300H přes 90 °C při špičkách): `.wslconfig` `processors=4` v sekci `[wsl2]` (VM dostane polovinu vláken, Docker buildy a testy nesaturují celý procesor), compose `cpus: 2.0` engine / `1.5` news-engine (tlumí jen nárazy, v klidu služby berou jednotky %), vitest `maxWorkers: 2` ; testovací sady a buildy pouštět po jednom, ne paralelně ve více worktree. **Pozor na sekce** `.wslconfig` : `autoMemoryReclaim=gradual` a `sparseVhd=true` patří do `[experimental]` — v `[wsl2]` je WSL 2.7 hlásí „Neznámá klávesa“ a ignoruje (12. 9. 2026 se tak zjistilo, že bod B z #1105 doted' neplatil; ověření: `wsl -d docker-desktop -e true` nesmí vypsát varování). Změny platí po `wsl --shutdown` (= restart stacku, jen v okně). Hygiena: dev stack (`compose.dev.yml`) nikdy nenechávat běžet vedle produkce, prohlížeč s heatmapou na jednom tabu — PC s 16 GB má po 6 GB pro Docker a IB Gateway málo rezervy.

Disk Dockeru (VHDX) a úklid (#1127). Datový disk Docker Desktopu

(`D:\Programy\Docker\DockerDesktopWSL\disk\docker_data.vhdx` , cesta v Settings → Resources) **jen**

roste: každý build přidá vrstvy do build cache a každý deploy rollback tag `gex-*:pre-<issue>` . 12. 9. 2026 měl 71 GB (43 GB build cache, 36 rollback tagů), build zaplnil disk D: na 0 B, containerd ve VM spadl a Docker Desktop se nerozjel („Reset to factory defaults“ NIKDY — pgdata je uvnitř VHDX). Proto:

- `scripts/docker-cleanup.ps1` maže jen rollback tagy (nechá 3 na službu), build cache starší 7 dnů a osiřelé image; volumes se nedotýká. Běží po každém úspěšném deployi (`deploy-engine-offhours.ps1` krok 5c) a týdně v sobotu 08:00 (Task Scheduler „GEXLens docker úklid“, `scripts/register-docker-cleanup-task.ps1`, log `data/logs/docker-cleanup.log`). Při < 15 GB volných na disku s VHDX nebo VHDX > 40 GB pošle alert `disk_low` do zvonku a skončí s exit 2.
- Smazaná data VHDX **sám nevrátí** — místo uvolní až kompakce: `pwsh -File scripts/compact-docker-vhdx.ps1` **jako správce** (diskpart), Docker při ní 2–5 min stojí → jen při zavřeném trhu; stack naběhne sám.
- Lokální build image jen nouzově a **po jedné službě** (`docker compose build engine`, pak `frontend`), nikdy všechny naráz, a před buildem zkontrolovat volné místo; standardně image dodává CI (#1139, kap. 3).

13.5 Ověření a denní provoz

```
Test-NetConnection 127.0.0.1 -Port 7496 # TcpTestSucceeded: True = API poslouchá (Gateway: 4001)
```

Každé pondělí ráno: potvrdit IB Key push v Gateway (týdenní reautentizace, kap. 13.2). Každý obchodní den: TWS/Gateway běží a je přihlášený **před startem enginu**; stavová lišta aplikace ukazuje `connected :7496` (Gateway `:4001`) a `• Live` (ne Offline). Diagnostika problémů: kap. 12.

Restart nebo nasazení enginu: vždy `pwsh scripts/deploy-engine-offhours.ps1` (pauza Globexu, rollback, **uložení logu před recreate** — #1056). Ruční `docker compose up -d engine` jen s předchozím `docker logs gex-engine-1 > data/logs/engine-<<YYYYMMDD-HHMM>.log 2>&1`; jinak důkazy o předchozím běhu zmizí.

14. Bezpečnost a nasazení na server

Výchozí stav aplikace počítá s tím, že běží na jednom PC za NATem. Přesun na VPS s veřejnou IP (#539) tenhle předpoklad ruší, proto proběhla prověrka #542. Tahle kapitola je její provozní výstup.

Tajemství

Dvě hodnoty musí být v lokálním `.env` (do repa nepatří, `.gitignore` je drží venku):

Proměnná	K čemu
GEXLENS_PG_PASSWORD	Heslo k PostgreSQL. Compose bez něj nenastartuje — žádný slabý default už neexistuje.
GEXLENS_API_TOKEN	Sdílené tajemství pro <code>/internal/*</code> (engine i news-engine → API) a <code>/backup/postgres</code> .

Vygeneruje je `pwsh scripts/init-secrets.ps1` — skript je idempotentní, existující hodnoty nepřepisuje a heslo rovnou přepíše i v běžícím PostgreSQL (`ALTER USER`). To je nutné: `POSTGRES_PASSWORD` se uplatní jen při prvním `initdb`, takže na existujícím volume by samotná změna v `.env` znamenala, že se služby k DB nepřipojí.

Token do UI patří jen kvůli stažení zálohy — vkládá se do pole **Settings** → **Záloha databáze** → **API token** a zůstává v localStorage prohlížeče. V image frontendu být nesmí, ten si stáhne kdokoli.

Síť

`compose.yml` nepublikuje nic na `0.0.0.0`. Adresu řídí `GEXLENS_BIND_ADDR` (default `127.0.0.1`); na serveru sem patří Tailscale IP. **Nikdy** `0.0.0.0` — Docker zapisuje publikaci do řetězce `DOCKER`, který se na Linuxu vyhodnocuje před UFW, takže port publikovaný bez adresy je veřejný i se „zapnutým firewallem“.

Prohlížeč mluví jen s nginx (`:8080`), který proxuje `/api` → `api:8000` včetně WebSocketu. Port API i PostgreSQL zůstávají publikované na loopback jen pro nástroje na hostiteli (zálohy, sondy) — pro provoz je potřeba nemá.

Checklist před nasazením na VPS

1. `pwsh scripts/init-secrets.ps1` na serveru; ověřit, že `.env` má práva 600.
2. `GEXLENS_BIND_ADDR` = Tailscale IP serveru.
3. `GEXLENS_ALLOWED_ORIGINS` = adresa UI, pod kterou se bude otevírat (jinak prohlížeč zablokuje fetch a WS handshake skončí na kontrole Origin).
4. UFW: povolit jen SSH a Tailscale; ověřit `nmap` z venku, že 8080/8000/55432 nejsou vidět (kontrolovat zvenčí, ne `ufw status` — viz past s DOCKER chainem).
5. SSH: klíče, `PasswordAuthentication no`, root login zakázaný.
6. IB Gateway: **VNC nikdy veřejně**, jen přes tunel.
7. Zálohy: `scripts/backup-postgres.ps1` (nebo cron s `pg_dump`) mimo server, šifrovaně — dump obsahuje celý OL archiv a historii setupů.

8. `docker compose up -d --build`, pak ověřit, že engine publikuje (`/status` ukazuje `engine: online`) — chybějící token se pozná tak, že UI zůstane bez živých dat a engine loguje chybu hned při startu.

Linux server: IB Gateway v kontejneru a rozdíly proti PC (#1094)

Postup zprovoznění krok za krokem (netcup RS 1000 G12, Tailscale, přenos dat, přepnutí v pauze Globexu) je v issue #1094. Co k tomu má repo:

Co	Kde	Poznámka
IB Gateway jako služba	<code>compose.server.yml</code>	vždy <code>-f compose.yml -f compose.server.yml</code> ; image <code>ghcr.io/gnzsanz/ib-gateway:stable</code> (IBC + Xvfb), API na <code>ib-gateway:4003</code> (socat relay — 4001 je uvnitř kontejneru jen localhost), nic nepublikuje, VNC vypnuté
Username a heslo	<code>.env</code> → <code>IBKR_DATA_USERID</code> ; <code>secrets/ibkr_password</code> (mimo git, <code>chmod 600</code>)	druhý (datový) username účtu (#737), nikdy hlavní — souběh s mobilem by shodil Gateway
Reautentizace	<code>AUTO_RESTART_TIME</code> 23:05, <code>TWS_COLD_RESTART</code> neděle 22:30 (TZ kontejneru Europe/Prague)	pondělní IB Key push zůstává ruční (kap. 13.2); <code>TWOFA_TIMEOUT_ACTION=restart</code> login opakuje, dokud push nepřijde
Past host/port v DB	Settings UI → host <code>ib-gateway</code> , port <code>4003</code>	hodnota z DB přebíjí <code>.env</code> i <code>compose</code> (#446/#992); po obnově dumpu z PC tam zůstal domácí <code>4001</code>
Název projektu	<code>compose.yml</code> → <code>name: gex</code>	kontejnery <code>gex-*</code> a image <code>gex-engine</code> nezávisle na názvu adresáře (<code>/srv/gexlens</code>) — deploy skript a rollback tag s nimi počítají
Deploy	<code>pwsh scripts/deploy-engine-offhours.ps1 -Server</code>	<code>-Server</code> přidá override; zóna Chicago se hledá jako <code>America/Chicago</code> i <code>Central Standard Time</code>
Walk-forward	<code>scripts/systemd/gexlens-walkforward.{service,timer}</code>	náhrada Task Scheduleru; po–pá 23:30, <code>Persistent=true</code>
Přístup k UI	<code>tailscale serve --bg --https=443 localhost:8080</code>	<code>GEXLENS_BIND_ADDR</code> zůstává loopback, <code>GEXLENS_ALLOWED_ORIGINS=https://<host></code> .

Co	Kde	Poznámka
		<tailnet>.ts.net ; UFW nepouští nic, SSH přes Tailscale
Docker × UFW	řetězec DOCKER-USER v /etc/ufw/after.rules zahazuje vše z veřejného rozhraní do kontejnerů	druhá pojistka vedle loopback bindu; kontrola vždy nmap zvenčí
Swap na 8 GB	zram-tools , PERCENT=25	naměřeno 5,3 GB (kap. 13.2 + docker stats 9. 9. 2026); zram kryje špičky RTH, práh pro větší plán je 85 %

Návrat zpět (rollback)

Rotace hesla nesahá na data — mění jen přihlašovací údaj, pgdata volume ani parquety se nedotkne. Vrátit ji lze kdykoli:

```
docker exec $(docker compose ps -q postgres) psql -U gexlens -d gexlens `
-c "ALTER USER gexlens PASSWORD 'gexlens';"
```

Funguje vždy, i když se heslo v .env rozejde s databází nebo .env ztratí: psql uvnitř kontejneru chodí přes unix socket, který heslo nevyžaduje. Stejným příkazem se dá nastavit i nová hodnota, když je potřeba sladit .env s běžícím serverem. Po změně restartovat stack (docker compose up -d) — běžící kontejnery drží spojení se starým heslem.

Při revertu změn #542 na tohle nezapomenout. Starší compose.yml má POSTGRES_PASSWORD: gexlens natvrdo, takže po návratu ke starému kódu se služby k databázi s rotovaným heslem nepřipojí. Pořadí: nejdřív vrátit heslo příkazem výše, pak revertovat kód.

Opakovaná prověrka

pwsh scripts/security-scan.ps1 spustí gitleaks (celá historie), pip-audit a npm audit; s -Images navíc trivy nad postavenými images. První tři běží i v CI jako job security na každý PR. Nálezy trivy jsou zpravidla OS balíky base image — řeší je rebuild s čerstvou bází, ne zásah do kódu, proto v CI nejsou.

Interní dokument. Uživatelská příručka: UZIVATELSKY-MANUAL.md (dostupná i v aplikaci jako Wiki).

14.9 Přepnutí repozitáře na soukromé — co se rozbije a co udělat

Repo je od září 2026 veřejné a část provozu na tom stojí (13. 9. 2026, #1139). Před přepnutím na *Private* (Settings → General → Danger Zone → Change visibility) projít tento seznam; body 1–3 jsou povinné, jinak přestane fungovat nasazení nebo ochrana `main`.

co	dnes (veřejné)	po přepnutí (soukromé, plán GitHub Free)	co udělat
Image v GHCR (<code>gex-python</code> , <code>gex-frontend</code>)	veřejné balíčky, zdarma, pull bez přihlášení	balíček navázaný na repo zůstane veřejný (viditelnost balíčku je nezávislá) — pokud ho přepneš na private, platí limit 500 MB úložiště + 1 GB přenosu/měs. , což dva image (~360 MB komprimovaně) s tagy <code>sha-*</code> přerostou během týdne	1. Bud' nechat balíčky veřejné (kód je z image stejně dohledatelný jen jako bytecode; tajemství v nich nejsou, kap. 14.1), nebo při private: přidat do workflow <i>Images</i> krok <code>actions/delete-package-</code> <code>versions</code> (nechat <code>latest</code> + 3 poslední <code>sha-*</code>) a na každém stroji, který táhne image, <code>docker login</code> <code>ghcr.io</code> (PAT classic s jen <code>read:packages</code> , uložit do <code>.env</code> jako <code>GHCR_TOKEN</code> , nikdy do repa; <code>echo</code> <code>\$env:GHCR_TOKEN docker</code> <code>login ghcr.io -u kEchiCZ --</code> <code>password-stdin</code>)
GitHub Actions (CI + Images)	minuty zdarma bez limitu	2 000 min/měs. zdarma; CI ~5 min na PR + Images ~6 min na merge ⇒ ~10–15 min na PR, tj. cca 130 PR/měs. v limitu	2. Sledovat Settings → Billing → Actions; při přiblížení limitu vypnout Images pro PR (dnes běží jen na main — OK) a sloučit CI joby

co	dnes (veřejné)	po přepnutí (soukromé, plán GitHub Free)	co udělat
Ruleset <code>main</code> (PR + zelené CI + jen squash, žádný force push)	vymáhá GitHub	rulesety a branch protection na soukromém repu vyžadují GitHub Pro — na Free se přestanou vymáhat (zůstanou uložené, ale neaktivní)	3. Buď GitHub Pro (~4 \$/měs.), nebo se spolehnout na <code>scripts/merge-when- green.sh</code> (merguje jen po zeleném CI) a na kázeň: nikdy <code>git push origin main</code> přímo. Automatické mazání větví po merge funguje dál
Dependabot, gitleaks, pip- audit, npm audit	běží	běží stejně (Dependabot i CI jsou na Free pro soukromá repa)	nic
Odkazy na soubory/obrázky v issues a wiki	veřejné URL <code>raw.githubusercontent.com</code>	fungují jen přihlášeným	nic (wiki v aplikaci má obrázky u sebe v <code>frontend/public/manual/img</code>)
<code>gh</code> CLI v tomto počítači	token <code>repo</code> stačí	totéž; pro balíčky navíc <code>read:packages</code> (bod 1)	<code>gh auth refresh -s read:packages</code> jen při private balíčcích

Co se **nemění**: tajemství zůstávají výhradně v `.env` (kap. 14.1), image je neobsahují, `docker compose pull` + deploy skript fungují beze změny, pokud zůstanou balíčky veřejné. Pořadí při přepnutí: nejdřív rozhodnout bod 1 (balíčky), pak bod 3 (ochrana `main`), teprve pak přepnout viditelnost repa a ověřit `docker compose pull frontend` na produkci.